

CS2 AI COACHING SYSTEM

Ultimate CS2 Coach

Parte 1A — Il Cervello

Architettura neurale, addestramento, JEPA, VL-JEPA, Superposition Layer, LSTM + Mixture of Experts

Autore: Renan Augusto Macena

Marzo 2026

Ultimate CS2 Coach — Parte 1A: Il Cervello

Argomenti: Architettura neurale completa, regime di addestramento, modelli di deep learning (JEPA, VL-JEPA, LSTM+MoE), Osservatorio di Introspezione del Coach, contratto 25-dim, principio NO-WALLHACK, e panoramica dell'architettura di sistema (avvio, interfaccia desktop Qt/PySide6, architettura quad-daemon, pipeline generale).

Autore: Renan Augusto Macena

Introduzione di Renan

Questo progetto è il risultato di innumerevoli ore di preparazione, perfezionamento e, soprattutto, ricerca. Counter-Strike è il mio calcio, qualcosa di cui sono appassionato da tutta la vita, come la musica. In questo gioco ho ormai più di 10 mila ore e ci gioco dal 2004, ho sempre desiderato una guida professionale, come quella dei veri giocatori professionisti, per capire come appare realmente quando qualcuno si allena nel modo giusto, e gioca nel modo giusto sapendo cosa è giusto o sbagliato e non solo supponendolo. A questo punto la mia fiducia è legata alla mia conoscenza del gioco, ma è ancora lontana dal livello a cui vorrei giocare. Immagino che molte persone come me, in questo gioco o forse in altri, si sentano allo stesso modo: fanno di avere esperienza e conoscenza di ciò che stanno facendo, ma si rendono conto che i giocatori professionisti sono così tanto più abili e rifiniti che persino anni di esperienza non possono eguagliare una frazione di ciò che questi professionisti offrono.

Questo progetto tenta di "dare vita", utilizzando tecniche avanzate, a un coach definitivo per Counter-Strike, uno che capisca il gioco, valuti dozzine di aspetti con chiarezza e giudizio raffinato, assimili e diventi più intelligente col passare del tempo.. Il mio obiettivo finale è farlo ingerire, elaborare e imparare da tutte le partite pro di sempre, non tutti i playoff, oltre ad essere irrealistico per le dimensioni e il tempo di elaborazione, sarebbe inutile poiché il mio obiettivo è avere questo coach definitivo basato sul meglio del meglio. Il modello per il giocatore "a bassa abilità" sarà sempre l'utente, lui o lei non avrà mai demo e partite da cui questo coach possa imparare; invece, questo è un metodo di approccio completamente diverso, poiché il coach utilizzerà il suo modello di alta qualità per confrontarlo con il modello dell'utente, mostrando quanto l'utente sia realmente lontano dal livello di un giocatore professionista e, cosa più importante, adattando gli insegnamenti per aiutare l'utente a raggiungere il livello pro, adattandosi a ogni diverso stile di gioco: se sono un awper, lentamente ma inesorabilmente questo coach mi insegnerà come diventare un awper professionista, e questo vale per ogni ruolo nel gioco in cui un utente voglia diventare un professionista.

E ora con la passione che sto sviluppando per la programmazione, capendo tutta questa roba complicata sulla gestione del database, SQL, modelli di machine learning, Python, e quanto possa essere elegante, da tutti questi passaggi tecnici di implementazione, regolazione e creazione di API, e capendo cos'è quella .dll che ho visto per tutta la vita dentro le cartelle del gioco, capendo a cosa servono quei framework come Kivy per creare la grafica, imparando cosa significa realmente creare software, ho dovuto imparare ad andare su Linux (non sono decisamente un professionista, e se non avessi seguito le stesse istruzioni che ho creato mentre facevo ricerche e continuavo a lavorare su questo progetto, non ci sarei riuscito) per capire come costruire il programma, come renderlo cross-platform. Da Linux, ho costruito la versione APK (devo finire di lavorare anche su quella), e ho almeno capito i concetti più importanti della Compilazione, poi ho finito la build desktop su Windows e ho fatto tutto il resto, compilazione e packaging. In breve, spingersi al limite per capire, sfidare se stessi così tanto, su così tanti aspetti diversi che devi o assimilare non solo qualcosa ma anche le specifiche e il quadro complesso di tutto, o non arriverai da nessuna parte.

Ho usato strumenti, come Claude CLI sul terminale windows e linux, per aiutarmi a organizzare il codice e correggere gli errori di sintassi. Ho creato una cartella nella cartella del progetto contenente una "Tool Suite" composta da script Python che ho aiutato a creare e che hanno una tonnellata di funzioni utili, dal debug alla modifica di valori, regole, funzioni, stringhe a livello globale o locale del progetto in modo che da un singolo input all'interno di quello strumento, io possa cambiare tutto ciò che voglio "ovunque", persino cambiare le cose sulla dashboard grafica con gli input. Non dormo da circa venti giorni consecutivi (dal 24 dicembre 2025), sì, ma penso che ne valga la pena, e so bene che questo è solo un allenamento alla fine che ho capito come fare da solo dall'inizio alla fine, quindi ci sono certamente errori in giro. Se possa essere davvero utile, davvero funzionale, ecc., ecc., e molte altre belle cose, non lo so. Non voglio sopravvalutare quello che sto facendo. Lo sto facendo, ma ci ho messo davvero molto impegno.

Spero che qualcosa lì dentro possa essere utile.

Nota: Il framework UI è stato successivamente migrato da Kivy a Qt/PySide6 (marzo 2026), come documentato nella Parte 3.

Indice

Parte 1A — Il Cervello: Architettura Neurale e Addestramento (questo documento)

1. [Riepilogo esecutivo](#)
2. [Panoramica dell'architettura di sistema](#)
 - Principio NO-WALLHACK e Contratto 25-dim
3. [Sottosistema 1 — Core della rete neurale \(\[backend/nn/\]\(#\) \)](#)
 - AdvancedCoachNN (LSTM + MoE)
 - JEPA (Auto-Supervisionato InfoNCE)
 - **VL-JEPA** (Visione-Linguaggio, 16 Concetti di Coaching, ConceptLabeler)
 - JEPATrainer (Addestramento + Monitoraggio Deriva)
 - Pipeline Standalone (jepa_train.py)
 - SuperpositionLayer (Gating Contestuale, Osservabilità Avanzata, Inizializzazione Kaiming)
 - Modulo EMA
 - CoachTrainingManager, TrainingOrchestrator, ModelFactory, Config
 - NeuralRoleHead (Classificazione Ruoli MLP)
 - Coach Introspection Observatory (MaturityObservatory — Macchina a 5 Stati)

Parte 1B — I Sensi e lo Specialista: Modello RAP Coach (architettura 7 componenti, ChronovisorScanner, GhostEngine), Sorgenti Dati (Demo Parser, HLTv, Steam, FACEIT, TensorFactory, FAISS)

Parte 2 — Sezioni 5-13: Servizi di Coaching, Coaching Engines, Conoscenza e Recupero, Motori di Analisi (11), Elaborazione e Feature Engineering, Modulo di Controllo, Progresso e Tendenze, Database e Storage (Tri-Tier), Pipeline di Addestramento e Orchestrazione, Funzioni di Perdita

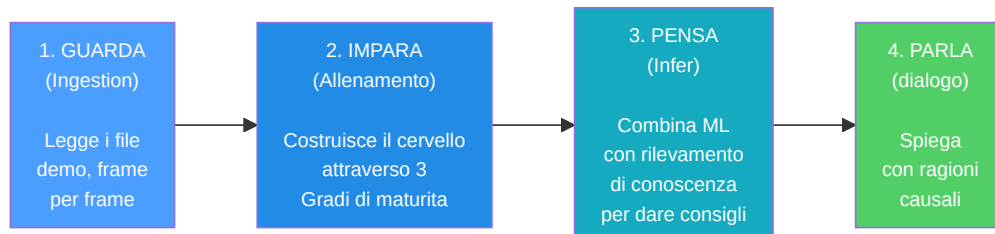
Parte 3 — Logica Programma, UI, Ingestion, Tools, Tests, Build, Remediation

1. Riepilogo esecutivo

CS2 Ultimate è un **sistema di coaching basato su IA ibrido** per Counter-Strike 2 (CS2). Combina modelli di deep learning (JEPA, VL-JEPA, LSTM+MoE, un'architettura RAP a 7 componenti (Percezione, Memoria LTC+Hopfield, Strategia, Pedagogia, Attribuzione Causale, Testa di Posizionamento + Comunicazione esterna), un Neural Role Classification Head), un Coach Introspection Observatory, un Retrieval-Augmented Generation (RAG), la banca dati delle esperienze COPER, la ricerca basata sulla teoria dei giochi, la modellazione bayesiana delle credenze e un'architettura Quad-Daemon (Hunter, Digester, Teacher, Pulse) in una pipeline unificata che:

1. **Ingerisce** file demo professionali e utente, estraendo statistiche di stato a livello di tick e di partita.
2. **Addestra** più modelli di reti neurali attraverso un programma a fasi con limiti di maturità (a 3 livelli: CALIBRAZIONE → APPRENDIMENTO → MATURO). 3. **Inferisce** consigli di allenamento fondendo le previsioni di apprendimento automatico con conoscenze tattiche recuperate semanticamente tramite una catena di fallback a 4 livelli (COPER → Ibrido → RAG → Base).
3. **Spiega** il suo ragionamento tramite attribuzione causale, narrazioni basate su template, confronti tra giocatori professionisti e un'opzionale rifinitura LLM (Ollama).

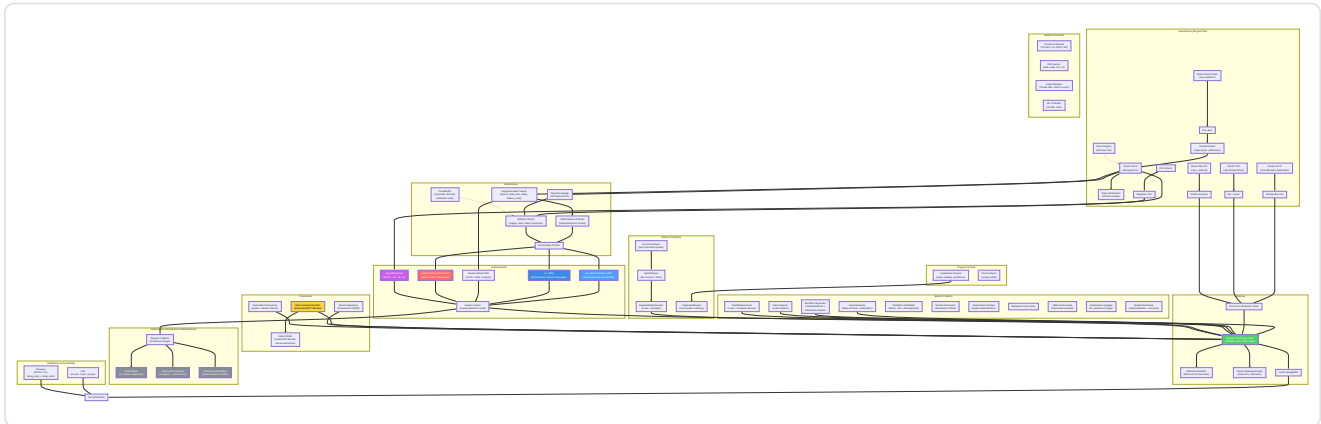
Il sistema contiene **≈ 103.600 righe di Python** distribuite su 445 file `.py` sotto `Programma_CS2_RENAN/`, che si estendono su **otto sottosistemi logici AI** (NN Core con VL-JEPA, RAP Coach + RAP Lite, Coaching Services, Knowledge & Retrieval, Analysis Engines (11), Processing & Feature Engineering, Sorgenti Dati, Motori di Coaching), un Osservatorio di addestramento, un modulo di Controllo (Console con REST API, DB Governor, Ingest Manager, ML Controller), un'architettura Quad-Daemon per l'automazione in background (Hunter, Digester, Teacher, Pulse), un'interfaccia desktop Qt/PySide6 con 15 schermi e pattern MVVM (migrata da Kivy nel marzo 2026), un sistema completo di ingestione (con sottosistema HLTV dedicato: HLTVApiService, CircuitBreaker, RateLimiter), storage e reporting, una **Tools Suite** con 41 script Python di validazione e diagnostica (29 root + 12 nel pacchetto — Goliath Hospital, headless validator, Ultimate ML Coach Debugger, `validate_coaching_pipeline`, `ingest_pro_demos`, `dead_code_detector`, `dev_health`, `rebuild_monolith`, `tick_census`), un'architettura tri-database specializzata con 18 tabelle SQLModel nel monolite (+ 3 nel database HLTV separato + 3 nei database per-match = 24 totali), e una **Test Suite** con 130 file di test organizzati in 6 categorie: analysis/theory, coaching/training, ML/models, data/storage, UI/playback, integration/misc. Il progetto ha attraversato un processo di **rimediazione sistematica in 13 fasi** che ha risolto 412+ problemi di qualità del codice, correttezza ML, sicurezza e architettura, inclusa l'eliminazione di label leakage nell'addestramento (G-01), l'implementazione della zona di pericolo visiva nel tensore vista (G-02), la calibrazione automatica dello stimatore bayesiano (G-07), e la correzione del fallback del coaching COPER (G-08), seguita da una **seconda ondata di rimediazione** che ha risolto ulteriori 162 problemi (31 HIGH + 131 MEDIUM) relativi a thread safety, schema drift, Qt lifecycle, e hardening dell'osservabilità, e da una **terza ondata** (aprile 2026) che ha affrontato 40+ problemi aggiuntivi inclusi type safety (SA-14–SA-27), dependency pinning (DEP-1), checkpoint security (CTF-1/2), DataLineage audit trail (DL-1), e correzioni UI/UX (UX-1/2/3). La pipeline end-to-end è stata completata il 12 marzo 2026: 11 demo professionali ingerite, 17.3M righe tick, 6.4GB database, JEPA pre-addestrato (train loss 0.9506, val loss 1.8248). Aggiornamento aprile 2026: 156 database per-match, AdvancedCoachNN completamente addestrato (`latest.pt`), database HLTV popolato con **161 giocatori professionisti reali** (32 team, 156 stat card), HybridCoachingEngine potenziato con selezione automatica del pro di riferimento per nome nei feedback di coaching, **Coach Book v4** espanso a **502 voci** di conoscenza tattica in 8 file JSON (7 mappe + generale) con 13 categorie, **CoachingDialogueEngine** per coaching multi-turno con drill-down per-giocatore e per-round, **MovementQualityAnalyzer** (11° motore di analisi), **EloAugmentedPredictor** per probabilità vittoria con feature Elo, **PlusMinus rating metric**, potenziamenti COPER (incertezza TrueSkill, semantica CRUD, replay prioritizzato), codifica posizione relativa al bombsite, prioritizzazione demo per varianza di coaching con scoring qualità via modello Huber, e **CS2 Coach Bench** benchmark di valutazione a 200 domande.



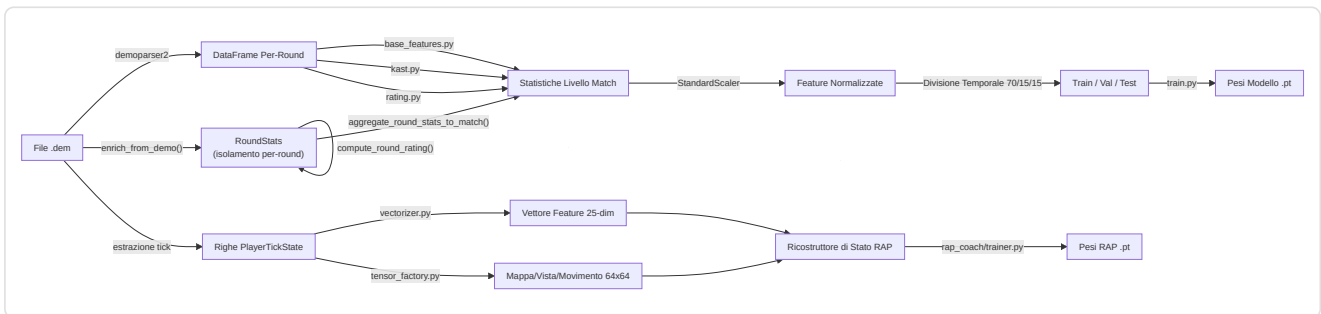
445 .py files · 102.000+ lines · 8 AI subsystems + Observatory + Control Module (REST API) + Quad-Daemon + Desktop UI Qt/PySide6 (15 screens) + 41 Tools (29 root + 12 inner) · 130 test files · 21 SQLModel tables (monolite) + 3 per-match · Architettura tri-database (database.db + hltv_metadata.db + database per-match) · Indicizzazione vettoriale FAISS (IndexFlatIP 384-dim) · Internazionalizzazione i18n (EN/IT/PT) · Accessibilità WCAG 1.4.1 (theme.py) · 12 rapporti di audit comprensivi (incl. revisione letteratura 140KB, 30 articoli peer-reviewed) · 610+ problemi risolti (412 in 13 fasi + 162 in seconda ondata + 40+ in terza ondata) · Pipeline end-to-end completata (156 per-match DB · JEPA + AdvancedCoachNN addestrati · 161 giocatori HLTV reali, 32 team, 156 stat card · Coach Book v4: 502 voci, 8 file, 13 categorie · CS2 Coach Bench: 200 domande)

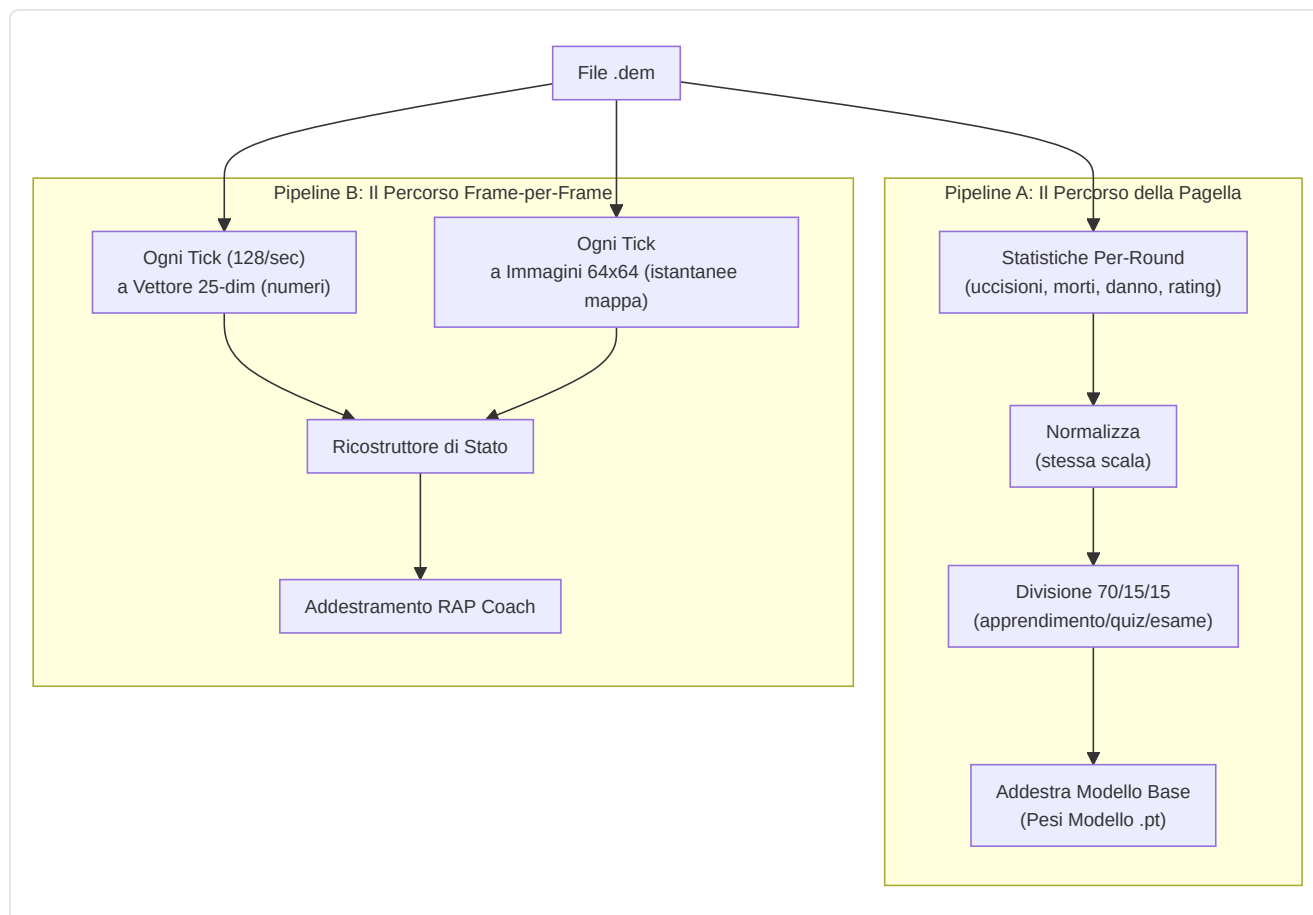
2. Panoramica dell'architettura del sistema

Il sistema è suddiviso in **6 sottosistemi principali** con responsabilità distinte. Ogni sottosistema ha un compito specifico e i dati fluiscono tra di essi in una pipeline ben definita.



Riepilogo flusso dei dati





Principio NO-WALLHACK e Contratto 25-dim

Due invarianti architetturali fondamentali attraversano l'intero sistema:

- 1. Principio NO-WALLHACK:** Il coach AI **vede solo ciò che il giocatore legittimamente conosce**. Quando il modulo `PlayerKnowledge` è disponibile, i tensori generati dalla `TensorFactory` codificano esclusivamente informazioni legittime: compagni di squadra (sempre visibili), nemici in posizioni "last-known" (con decadimento temporale, $\tau = 2.5s$), utilità propria e osservata. Nessuna informazione "wallhack" (posizioni nemiche reali non visibili) entra mai nel sistema di percezione. Quando `PlayerKnowledge` è `None`, il sistema ricade su una modalità legacy con tensori semplificati.
- 2. Contratto 25-dim (FeatureExtractor):** Il `FeatureExtractor` in `vectorizer.py` definisce il vettore di feature canonico a 25 dimensioni (`METADATA_DIM = 25`) usato da **tutti** i modelli (AdvancedCoachNN, JEPA, VL-JEPA, RAP Coach) sia in addestramento che in inferenza. Qualsiasi modifica al vettore di feature avviene **esclusivamente** nel `FeatureExtractor` — nessun altro modulo può definire feature proprie. Questo garantisce coerenza dimensionale end-to-end.

```

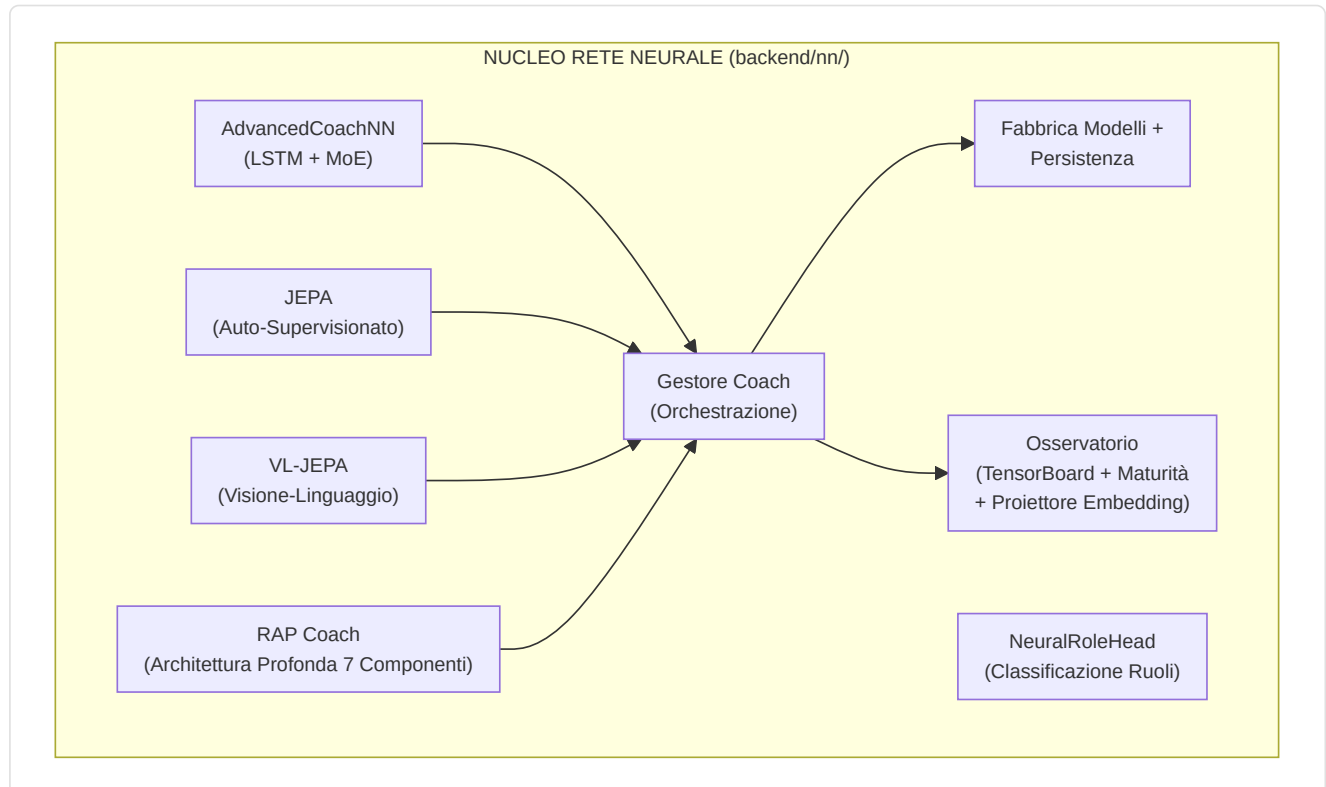
0: health/100      1: armor/100      2: has_helmet      3: has_defuser
4: equip/10000     5: is_crouching   6: is_scoped       7: is_blinded
8: enemies_vis     9: pos_x/4096     10: pos_y/4096     11: pos_z/1024
12: view_x_sin     13: view_x_cos    14: view_y/90      15: z_penalty
16: kast_est       17: map_id        18: round_phase
19: weapon_class   20: time_in_round/115 21: bomb_planted
22: teammates_alive/4 23: enemies_alive/5 24: team_economy/16000
  
```

Normalizzazione posizione: ``np.clip(pos_x / cfg.pos_xy_extent, -1.0, 1.0)`` dove ``pos_xy_extent=4096.0`` (default, configurabile per mappa). Il clip a `[-1,1]` protegge da coordinate fuori range che produrrebbero feature `> 1.0` in moduli non normalizzati.

3. Sottosistema 1 — Nucleo della rete neurale

Cartella nel programma: `backend/nn/` **File chiave:** `model.py`, `jepa_model.py`, `jepa_train.py`, `jepa_trainer.py`, `coach_manager.py`, `training_orchestrator.py`, `config.py`, `factory.py`, `persistence.py`, `role_head.py`, `training_callbacks.py`, `tensorboard_callback.py`, `maturity_observatory.py`, `embedding_projector.py`, `dataset.py`, `data_quality.py`, `evaluate.py`, `train_pipeline.py`, `training_monitor.py`, `early_stopping.py`, `ema.py`, `training_controller.py`, `win_probability_trainer.py`, `training_config.py`

Questo sottosistema contiene tutti i modelli di rete neurale, il "cervello" del sistema di coaching. Include sei distinte architetture di modelli (AdvancedCoachNN, JEPA, VL-JEPA, RAP Coach, RAP Lite, NeuralRoleHead), un gestore di training, un Osservatorio di Introspezione del Coach e utilità per la creazione e la persistenza dei modelli.



-AdvancedCoachNN (LSTM + Mix di Esperti)

Definito in `model.py`, questo è il fondamento del coaching supervisionato.

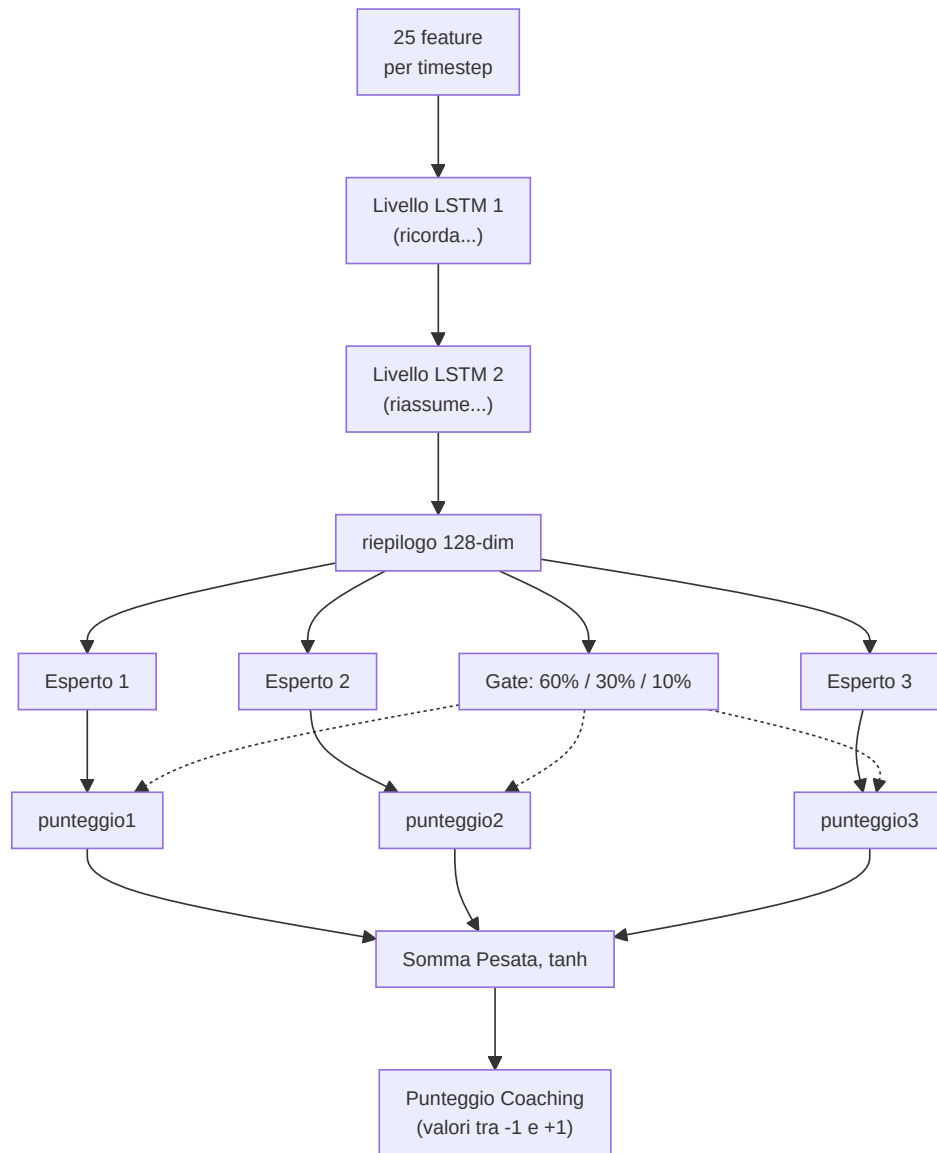
Componente	Dettaglio
Dimensione di input	25 funzionalità (<code>METADATA_DIM</code> da <code>vectorizer.py</code>)
Config	Dataclass <code>CoachNNConfig</code> : <code>input_dim=25</code> , <code>output_dim=METADATA_DIM</code> (default 25, ma sovrascritto a <code>OUTPUT_DIM=10</code> dalla <code>ModelFactory</code>), <code>hidden_dim=128</code> , <code>num_experts=3</code> , <code>num_lstm_layers=2</code> , <code>dropout=0.2</code> , <code>use_layer_norm=True</code>
Livelli nascosti	LSTM a 2 livelli (128 nascosti, <code>batch_first=True</code> , <code>dropout=0.2</code>) con <code>LayerNorm</code> post-LSTM
Testa dell'esperto	3 esperti lineari paralleli (configurabili), softmax-gated tramite una rete di gate appresa
Output	Somma pesata degli output degli esperti → vettore del punteggio di coaching a 10 dimensioni. La <code>CoachNNConfig</code> definisce <code>output_dim=METADATA_DIM</code> (25) come default, ma la <code>ModelFactory</code> sovrascrive con <code>OUTPUT_DIM=10</code> in produzione. L'alias <code>TeacherRefinementNN = AdvancedCoachNN</code> è mantenuto per retrocompatibilità
Bias di ruolo	Parametro <code>role_id</code> opzionale: <code>gate_weights = (gate_weights + role_bias) / 2.0</code> — orienta la selezione degli esperti verso conoscenze specifiche del ruolo
Validazione dell'input	<code>_validate_input_dim()</code> rimodella automaticamente 1D → <code>unsqueeze(0).unsqueeze(0)</code> e 2D → <code>unsqueeze(0)</code> per la robustezza

Ogni modulo esperto in `AdvancedCoachNN`: `Linear(128→128) → LayerNorm(128) → ReLU → Linear(128→output_dim)`.

Nota: `_create_expert()` di JEPA omette `LayerNorm` — solo `Linear → ReLU → Linear`. Si tratta di una scelta progettuale deliberata: gli esperti JEPA operano su incorporamenti latenti già normalizzati, mentre gli esperti `AdvancedCoachNN` elaborano output LSTM grezzi che traggono vantaggio dalla normalizzazione per esperto.

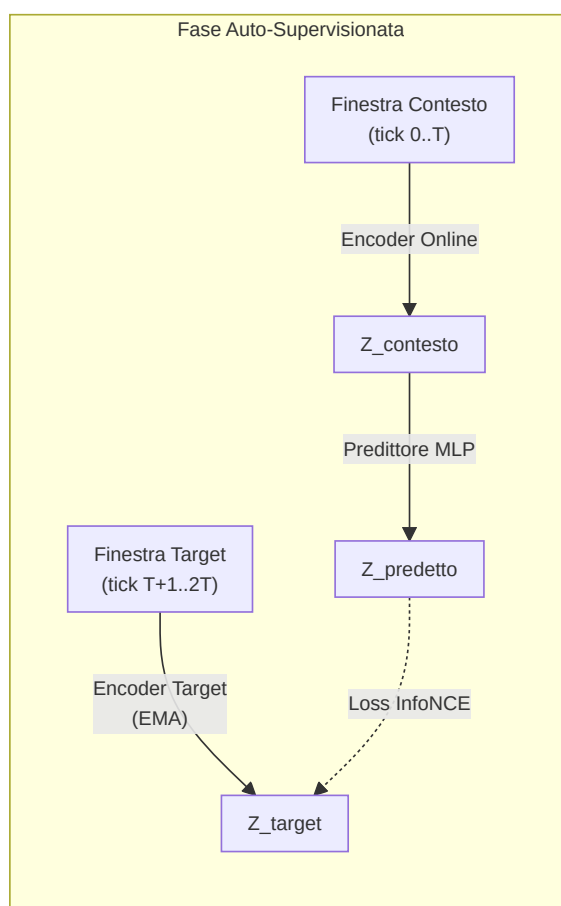
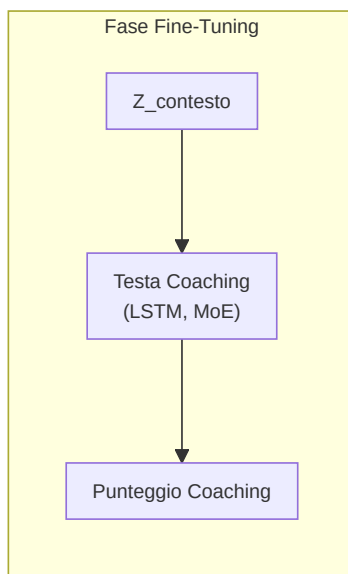
Passaggio in avanti (pseudo forward pass):

```
h, _ = LSTM(x) # x: [batch, seq_len, 25]
h = LayerNorm(h[:, -1, :]) # prendi l'ultimo timestep → [batch, 128]
gate_weights = softmax(W_gate · h) # [batch, 3]
expert_outputs = [E_i(h) for i in 1..3]
output = tanh(Σ gate_weights_i × expert_outputs_i)
```

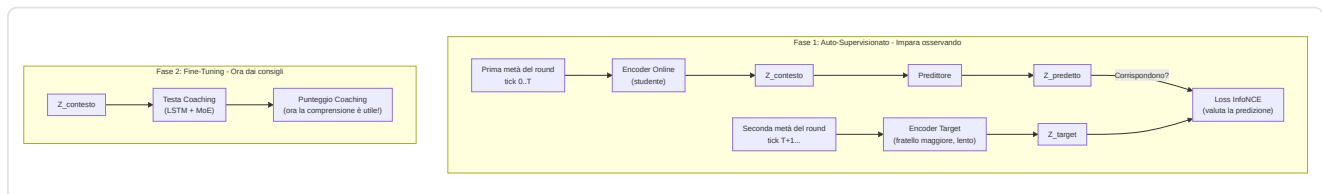
-Modello di coaching JEPA (Architettura predittiva con integrazione congiunta)

Definito in `jepa_model.py`. Un modello di **pre-allenamento auto-supervisionato** ispirato all'I-JEPA di Yann LeCun, adattato per dati CS2 sequenziali.



Dettagli dell'architettura:

Modulo	Parametri
Codificatore online	Linear(input_dim, 512) → LayerNorm → GELU → Dropout(0.1) → Linear(512, latent_dim=256) → LayerNorm
Codificatore target	Strutturalmente identico; aggiornato tramite media mobile esponenziale ($\tau = 0.996$). <code>EMA.state_dict()</code> restituisce tensori clonati per prevenire aliasing (un bug precedente permetteva la modifica accidentale dei pesi target attraverso riferimenti condivisi)
Predictor	Linear(256, 512) → LayerNorm → GELU → Dropout(0.1) → Linear(512, 256)
Coaching Head	LSTM(256, hidden_dim, 2 layers, dropout=0.2) → 3 esperti MoE → output controllato



Procedura di pre-addestramento (`jepa_trainer.py`):

- Carica le sequenze di `PlayerTickState` dai file SQLite demo professionali.
- Divide ogni sequenza in finestre di contesto e target.
- Codifica il contesto tramite il codificatore online + predittore, codifica il target tramite il codificatore del target (EMA).
- Riduce al minimo la **perdita di contrasto di InfoNCE** utilizzando negativi in batch con similarità del coseno e temperatura $\tau=0.07$.
- Dopo ogni batch, esegue l'aggiornamento EMA: $\theta_{\text{target}} \leftarrow \tau \cdot \theta_{\text{target}} + (1-\tau) \cdot \theta_{\text{online}}$.
- Monitoraggio della deriva:** Traccia gli oggetti DriftReport; attiva il riaddestramento automatico se la deriva $> 2.5\sigma$.
- Etichette basate sull'esito (Correzione G-01):** Il `ConceptLabeler` nell'addestramento VL-JEPA ora genera etichette dai dati `RoundStats` (esiti per round: uccisioni, morti, danni, sopravvivenza) anziché da feature a livello di tick. Questo elimina il **label leakage** — il problema precedente in cui le etichette dei concetti venivano derivate dalle stesse feature usate come input, permettendo al modello di "barare" durante l'addestramento senza apprendere effettivamente i pattern. Il metodo `label_from_round_stats(rs)` produce un vettore di 16 etichette di concetto basate su esiti misurabili. Se i dati `RoundStats` non sono disponibili, il sistema ricade sull'euristica legacy con un avviso di log una tantum.

Decodifica Selettiva (`forward_selective`): salta l'intero passaggio in avanti se la distanza del coseno tra l'embedding corrente e quello precedente è inferiore a una soglia (`skip_threshold=0.05`). Utilizza `1.0 - F.cosine_similarity()` come metrica di distanza e, durante l'operazione di salto, restituisce l'output precedente memorizzato nella cache. Questo consente un'efficace inferenza in tempo reale con salto dinamico dei frame: durante i momenti di gioco statici (i giocatori mantengono gli angoli), la maggior parte dei frame viene saltata completamente.

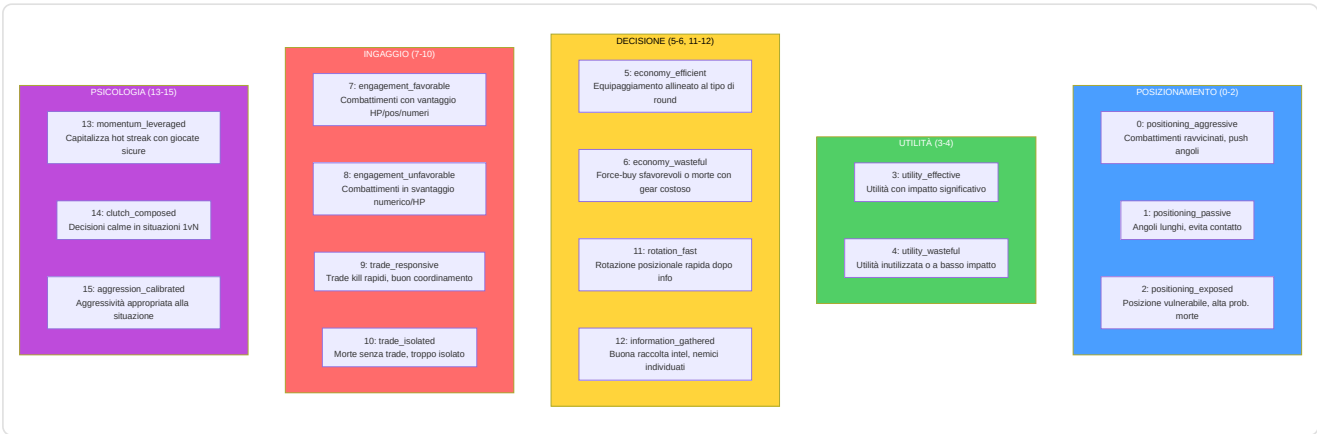
Adattatore JEPA per il coaching (`jepa_insight_adapter.py`): converte i vettori grezzi in uscita dalla Coaching Head in oggetti `InsightCandidate` pronti per la pipeline di coaching. Mappa i primi 10 elementi del contratto 25-dim a coppie messaggio aumento/diminuzione per area tattica (sopravvivenza, economia, movimento, utilità, posizionamento). L'uscita sigmoid è normalizzata a delta firmati `2*(out-0.5)`; delta con magnitudine inferiore a 0,1 vengono scartati come rumore. Il numero di candidati e il moltiplicatore di confidenza sono scalati dal livello di maturità: `doubt/crisis` → 0,5×, 1 candidato; `learning` → 0,8×, 2 candidati; `conviction/mature` → 1,0×, 3 candidati. Attivato dalla flag `USE_JEPA_MODEL` (disabilitata per default, F1.2 — comportamento byte-identico all'attuale quando spenta). Rispetta il vincolo NO-WALLHACK: consuma esclusivamente la finestra tick del giocatore osservato (contratto POV 25-dim, P-X-01).

-VL-JEPA: Architettura di Allineamento Visione-Linguaggio con Concetti di Coaching

Definito nella seconda metà di `jepa_model.py`. Il VL-JEPA (**V**ision-**L**anguage **J**EPA) è un'estensione fondamentale del `JEPACoachingModel` che aggiunge un **meccanismo di allineamento tra embedding latenti e concetti di coaching interpretabili**. Ispirato al VL-JEPA di Meta FAIR (2026), mappa le rappresentazioni latenti in uno spazio concettuale strutturato con 16 concetti di coaching predefiniti.

Tassonomia dei 16 Concetti di Coaching

Il sistema definisce `NUM_COACHING_CONCEPTS = 16` concetti organizzati in **5 dimensioni tattiche**:



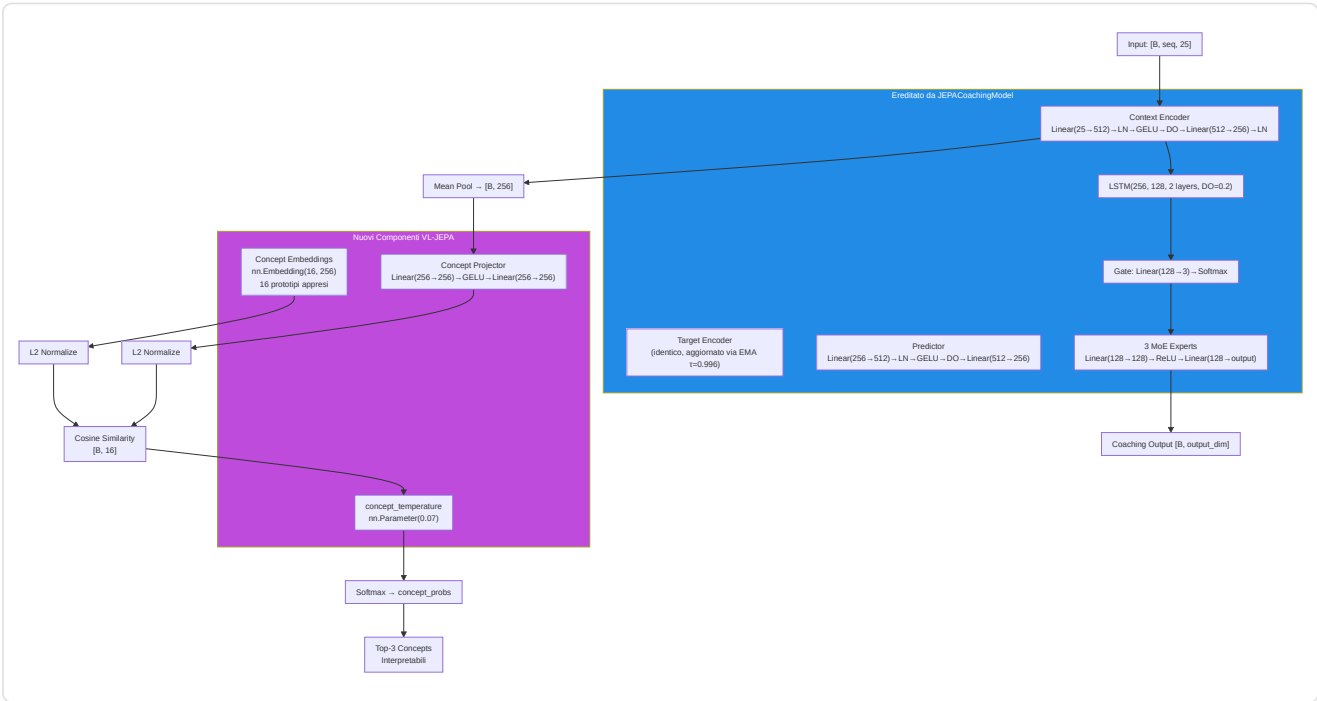
Ogni concetto è definito come un `CoachingConcept` dataclass immutabile con `(id, name, dimension, description)`. La lista globale `COACHING_CONCEPTS` e `CONCEPT_NAMES` sono le sorgenti di verità per tutto il sistema.

Architettura `VLJEPACoachingModel`

`VLJEPACoachingModel` eredita da `JEPACoachingModel` e aggiunge 3 componenti:

Componente	Parametri	Scopo
<code>concept_embeddings</code>	<code>nn.Embedding(16, latent_dim=256)</code>	16 prototipi di concetto appresi nello spazio latente
<code>concept_projector</code>	<code>Linear(256→256) → GELU → Linear(256→256)</code>	Proietta embedding encoder nello spazio allineato ai concetti
<code>concept_temperature</code>	<code>nn.Parameter(0.07)</code> , clamped <code>[0.01, 1.0]</code>	Temperatura appresa per scaling della similarità coseno

Tutti i percorsi forward del genitore (`forward`, `forward_coaching`, `forward_selective`, `forward_jepa_pretrain`) sono **preservati immutati** via ereditarietà. Il nuovo percorso `forward_vl()` aggiunge l'allineamento concettuale.



Percorso Forward VL-JEPA (forward_vl)

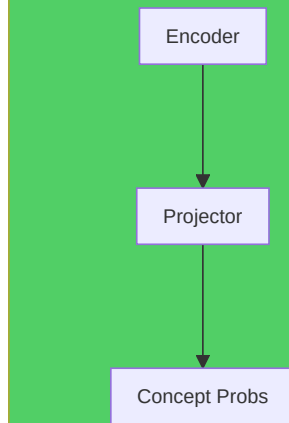
```
1. Encode:      embeddings = context_encoder(x)                # [B, seq, 256]
2. Pool:        latent = embeddings.mean(dim=1)                # [B, 256]
3. Project:     projected = L2_normalize(concept_projector(latent)) # [B, 256]
4. Similarity:  logits = projected @ concept_embs_norm.T      # [B, 16]
5. Temperature: probs = softmax(logits / clamp(temp, 0.01, 1.0)) # [B, 16]
6. Coaching:    coaching_output = forward_coaching(x, role_id) # [B, output_dim]
7. Decode:      top_concepts = top-k(probs, k=3)               # interpretabili
```

Output forward_vl() : Dizionario con 5 chiavi:

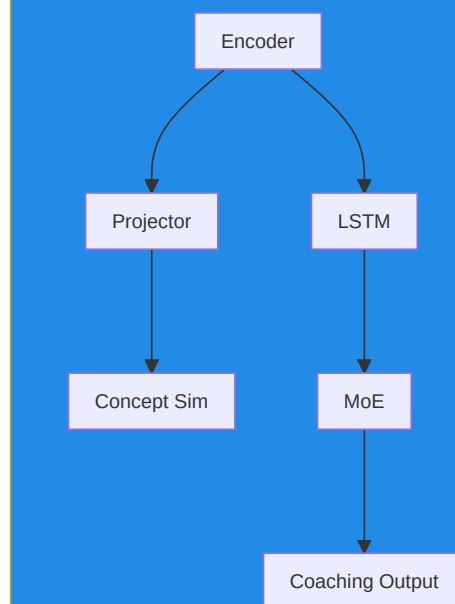
Chiave	Forma	Contenuto
concept_probs	[B, 16]	Probabilità softmax per ogni concetto
concept_logits	[B, 16]	Punteggi di similarità grezzi (pre-softmax)
top_concepts	List[tuple]	[(nome_concetto, probabilità), ...] per il primo campione
coaching_output	[B, output_dim]	Predizione coaching standard (via genitore)
latent	[B, 256]	Embedding latente pooled dell'encoder

Percorso leggero — get_concept_activations() : Forward solo concettuale senza testa di coaching né LSTM. Utilizza torch.no_grad() per efficienza massima durante l'inferenza.

get_concept_activations() — Solo Concetti



forward_vl() — Percorso Completo



ConceptLabeler: Due Modalità di Etichettatura

La classe `ConceptLabeler` genera **etichette soft multi-label** ($[0, 1]^{16}$) per l'addestramento VL-JEPA. Supporta due modalità:

Modalità 1 — Basata su Esiti (preferita, correzione G-01): `label_from_round_stats(round_stats)` genera etichette da dati di **esito del round** (uccisioni, morti, danni, sopravvivenza, trade kill, utilità, equipaggiamento, round vinto). Questi dati sono **ortogonali** al vettore di input a 25 dim, eliminando il label leakage.

Concetto	Segnale di Esito Usato
<code>positioning_aggressive</code> (0)	<code>opening_kill=True</code> → 0.8, <code>kills≥2+survived</code> → 0.6
<code>positioning_passive</code> (1)	<code>survived</code> , <code>no opening</code> , <code>damage<60</code> → 0.7
<code>positioning_exposed</code> (2)	<code>opening_death=True</code> → 0.8, <code>deaths>0+damage<40</code> → 0.6
<code>utility_effective</code> (3)	<code>utility_total>80</code> + <code>round_won</code> → 0.5+util/300
<code>utility_wasteful</code> (4)	<code>zero utility</code> → 0.5, <code>utility+lost</code> → 0.4
<code>economy_efficient</code> (5)	<code>eco win (equip<2000)</code> → 0.9, <code>normal win</code> → 0.7
<code>economy_wasteful</code> (6)	<code>high equip+loss</code> → 0.4+equip/16000
<code>engagement_favorable</code> (7)	<code>multi-kill+survived</code> → 0.5+kills×0.15
<code>engagement_unfavorable</code> (8)	<code>deaths+no kills+low dmg</code> → 0.7
<code>trade_responsive</code> (9)	<code>trade_kills>0</code> → 0.6+tk×0.2
<code>trade_isolated</code> (10)	<code>died</code> , <code>not traded</code> , <code>no trade kills</code> → 0.7
<code>rotation_fast</code> (11)	<code>assists≥1+round_won</code> → 0.6+assists×0.1
<code>information_gathered</code> (12)	<code>flashes≥2+survived</code> → 0.6
<code>momentum_leveraged</code> (13)	<code>rating>1.5</code> → <code>rating/2.5</code> , <code>kills≥3</code> → 0.7
<code>clutch_composed</code> (14)	<code>kills≥2+survived+won</code> → 0.6
<code>aggression_calibrated</code> (15)	<code>efficiency = kills×1000/equip</code> → <code>min(eff×0.5, 1.0)</code>

Modalità 2 — Euristica Legacy (fallback con label leakage): `label_tick(features)` genera etichette direttamente dal vettore di feature a 25 dim. Questo crea **label leakage** perché il modello può "barare" ricostruendo le feature di input anziché imparare pattern latenti. Usato solo quando `RoundStats` non è disponibile, con un avviso di log una tantum.

`label_batch(features_batch)`: Wrapper che gestisce batch 2D `[B, 25]` e 3D `[B, seq_len, 25]` (media delle etichette sulla sequenza per input 3D).

Riferimento indici feature (METADATA_DIM=25):

```
0: health/100      1: armor/100      2: has_helmet      3: has_defuser
4: equip/10000    5: is_crouching   6: is_scoped       7: is_blinded
8: enemies_vis    9: pos_x/4096     10: pos_y/4096     11: pos_z/1024
12: view_x_sin    13: view_x_cos    14: view_y/90      15: z_penalty
16: kast_est      17: map_id        18: round_phase
19: weapon_class  20: time_in_round/115  21: bomb_planted
22: teammates_alive/4  23: enemies_alive/5  24: team_economy/16000
```

Funzioni di Perdita VL-JEPA

1. `jepa_contrastive_loss()` — InfoNCE (già documentata sopra)

Formula: $-\log(\exp(\text{sim}(\text{pred}, \text{target})/\tau) / (\exp(\text{sim}(\text{pred}, \text{target})/\tau) + \sum \exp(\text{sim}(\text{pred}, \text{neg}_i)/\tau)))$ con $\tau=0.07$.

L'implementazione PyTorch utilizza un trucco efficiente: concatena la similarità positiva e le similarità negative in un vettore di logits `[pos_sim, neg_sim_1, ..., neg_sim_N]` e applica `F.cross_entropy(logits, labels=0)` — dove l'etichetta 0 indica che il primo elemento (la similarità positiva) è la classe corretta. Questo è matematicamente equivalente alla formula InfoNCE ma sfrutta la numericamente stabile `log_softmax` interna di PyTorch.

2. `vl_jepa_concept_loss()` — Allineamento Concetti + Diversità VICReg

```
concept_loss = BCE_with_logits(concept_logits, concept_labels) # Multi-label
diversity_loss = -std(L2_normalize(concept_embeddings), dim=0).mean() # VICReg
total = alpha * concept_loss + beta * diversity_loss
```

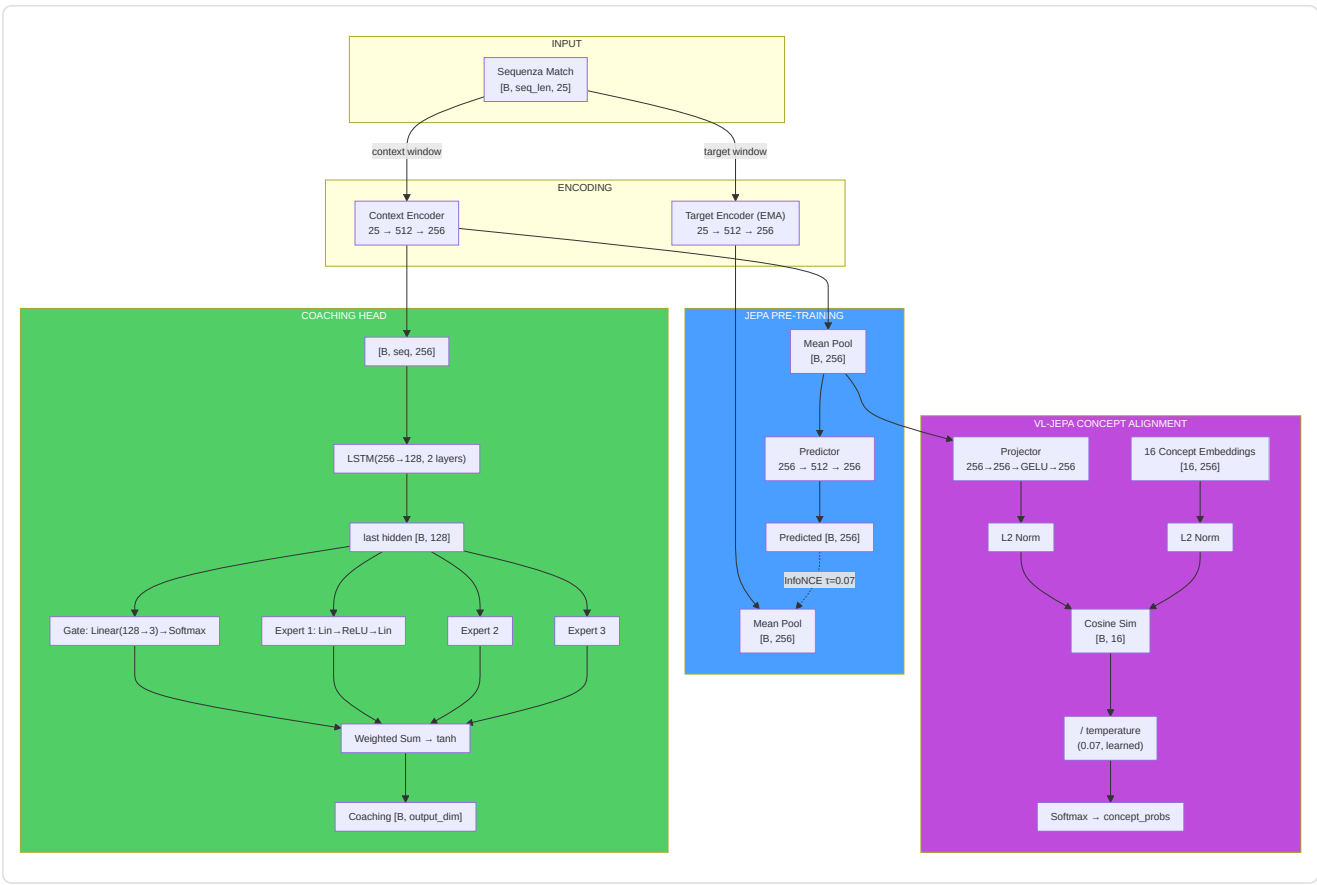
Termine	Formula	Peso Default	Scopo
concept_loss	<code>F.binary_cross_entropy_with_logits(logits, labels)</code>	$\alpha = 0.5$	Allinea embedding ai concetti corretti
diversity_loss	<code>-std_per_dim(L2_norm(concept_embs)).mean()</code>	$\beta = 0.1$	Impedisce il collasso degli embedding di concetto

Perdita totale nel training step VL-JEPA (`train_step_vl`):

```
L_total = L_inforce +  $\alpha$  × L_concept +  $\beta$  × L_diversity
```

Dove `L_inforce` è la perdita contrastiva standard JEPA e `( $\alpha$  × L_concept +  $\beta$  × L_diversity)` è il termine di allineamento concettuale.

Flusso Dimensionale Completo JEPA / VL-JEPA



JEPATrainer: Addestramento con Monitoraggio Deriva

Definito in `jepa_trainer.py` . Gestisce sia l'addestramento JEPA standard che VL-JEPA, con riaddestramento automatico basato sulla deriva.

Parametro	Default	Scopo
Optimizer	AdamW (lr=1e-4, weight_decay=1e-4)	Ottimizzazione con decadimento dei pesi
Scheduler	CosineAnnealingLR (T_max=100)	Decadimento ciclico del learning rate
DriftMonitor	z_threshold=2.5	Rileva drift delle feature oltre 2.5 σ
drift_history	<code>List[DriftReport]</code>	Storico dei report di drift

Codifica negativi condivisa — `encode_raw_negatives(negatives, seq_len)` (NN-H-02):

Metodo condiviso che codifica negativi grezzi (feature space) nello spazio latente. Quando l'orchestrator invia negativi dal cross-match pool (dimensione `METADATA_DIM`), questi devono essere trasformati in embedding latenti per il calcolo della loss contrastiva. Il metodo: reshape `[B*N, 1, D]` → expand per `seq_len` → encode via `target_encoder` con `torch.no_grad()` → mean pool → reshape `[B, N, latent_dim]`. Questa logica era precedentemente duplicata tra trainer e orchestrator; la centralizzazione (NN-H-02) previene disallineamenti.

Ciclo di addestramento — `train_step(x_context, x_target, negatives)`:

1. Forward pass JEPA: `pred, target = model.forward_jepa_pretrain(context, target)`
2. **Auto-detect negativi grezzi:** Se `negatives.shape[-1] != latent_dim`, i negativi sono feature grezze → vengono auto-codificati via `encode_raw_negatives()` (NN-H-02)
3. Loss InfoNCE su embedding normalizzati
4. Backward + optimizer step
5. **Aggiornamento EMA target encoder** (deve avvenire DOPO `optimizer.step()`)
6. **Monitoraggio diversità embedding (P9-02):** Calcola la varianza media delle dimensioni latenti. Se `variance < 0.01`, emette un warning di potenziale collasso degli embedding — il modello sta convergendo a una rappresentazione degenera dove tutti gli input producono lo stesso vettore

Guardia batch degenera (NN-JT-01): In modalità in-batch negatives, se `batch_size < 2` il batch viene saltato — non è possibile costruire negativi escludendo se stessi da un batch di un solo elemento. Questo previene errori nell'indicizzazione dei negativi durante le fasi iniziali dell'addestramento con pochi dati.

Training step VL-JEPA — `train_step_vl()`: Estende `train_step` con:

1. Forward pass JEPA standard (InfoNCE)
2. Forward VL: `model.forward_vl(x_context)` → `concept_logits`
3. **Generazione etichette (preferenza G-01):** Se `round_stats` è disponibile → `label_from_round_stats()` (no leakage). Altrimenti → `label_batch()` (euristica legacy con avviso una tantum)
4. Concept loss + diversity loss: `vl_jepa_concept_loss(logits, labels, embeddings, α, β)`
5. Loss totale: `L_infonce + L_concept_total`
6. Backward + optimize + EMA update

Output: `{total_loss, infonce_loss, concept_loss, diversity_loss}`.

Monitoraggio deriva — `check_val_drift(val_df, reference_stats)`:

- Utilizza `DriftMonitor` dalla pipeline di validazione
- Calcola z-score per ogni feature del validation set rispetto alle statistiche di riferimento
- Se `max_z_score > 2.5`, il report segna `is_drifted=True`
- `should_retrain(drift_history, window=5)` → se la maggioranza delle ultime 5 finestre mostra drift, attiva il flag `_needs_full_retrain`

Riaddestramento automatico — `retrain_if_needed(full_data_loader, device, epochs=10)`:

- Se il flag `_needs_full_retrain` è attivo, resetta il scheduler e riesegue `epochs` epoche complete
- Dopo il riaddestramento, cancella il flag e lo storico drift
- Restituisce `True/False` per indicare se il riaddestramento è avvenuto

Pipeline di Addestramento Standalone (`jepa_train.py`)

Script standalone per pre-addestramento e fine-tuning JEPA, eseguibile da CLI:

```
python -m Programma_CS2_RENAN.backend.nn.jepa_train --mode pretrain
python -m Programma_CS2_RENAN.backend.nn.jepa_train --mode finetune --model-path models/jepa_model.pt
```

JEPAPretrainDataset : Dataset PyTorch per il pre-addestramento:

Parametro	Default	Descrizione
<code>context_len</code>	10	Lunghezza finestra contesto (tick)
<code>target_len</code>	10	Lunghezza finestra target (tick)
<code>match_sequences</code>	<code>List[np.ndarray]</code>	Sequenze di match <code>[num_rounds, METADATA_DIM]</code>

Per ogni campione, seleziona un punto di partenza casuale nella sequenza e restituisce `{"context": [context_len, 25], "target": [target_len, 25]}`.

Nota (F3-25): Il punto di partenza usa `np.random.randint()` con stato globale non seedato → finestre non riproducibili tra run. Per addestramento deterministico, usare `worker_init_fn` o un `Generator` dedicato nel `DataLoader`.

`_roundstats_to_features(rs: RoundStats) → List[float]` : Estrae un vettore di **16 feature** da una singola riga `RoundStats`: `[kills, deaths, damage_dealt/100, headshot_kills, assists, trade_kills, was_traded, opening_kill, opening_death, he_damage/100, molotov_damage/100, flashes_thrown, smokes_thrown, equipment_value/5000, round_rating, side_CT]`. Il vettore viene poi paddato a `METADATA_DIM` (25) con zeri.

Esclusione `round_won` (P-RSB-03): Il campo `round_won` è **deliberatamente escluso** dal vettore feature. Includerlo causerebbe data leakage: il modello vedrebbe il risultato del round nei dati di input, permettendogli di predire banalmente gli esiti dal risultato stesso. `round_won` è correttamente utilizzato come **label** in `label_from_round_stats()` (`jepa_model.py`), dove genera le etichette di supervisione per il ramo VL-JEPA.

`_MIN_ROUNDS_FOR_SEQUENCE = 6` : Requisito minimo di round per costruire una sequenza di addestramento valida. Partite con meno di 6 round non forniscono abbastanza contesto temporale per apprendere pattern tattici significativi e vengono scartate silenziosamente.

`load_pro_demo_sequences(limit=100)` : Carica sequenze demo professionali dal database. Utilizza `_roundstats_to_features()` per estrarre feature reali per-round da `RoundStats`, con fallback a 12 feature aggregate a livello di match da `PlayerMatchStats` (paddate a `METADATA_DIM` con zeri) solo quando `RoundStats` non è disponibile.

⚠ Avvertimento Critico (F3-08): Nel percorso di fallback match-aggregate, lo script usa `np.tile(features, (20, 1))` per creare 20 frame identici da un singolo vettore aggregato. Questo rende il pre-addestramento JEPA **un'operazione identità** — il modello impara semplicemente a copiare l'input, non le dinamiche temporali. Il percorso primario con `RoundStats` reali e il `TrainingOrchestrator` nel percorso di produzione **non sono affetti** da questo problema e usano dati per-round/per-tick reali.

`train_jepa_pretrain()` : 50 epoche, `batch_size=16`, `lr=1e-4`, 8 negativi in-batch. L'optimizer include SOLO `context_encoder` e `predictor` — il `target_encoder` è aggiornato esclusivamente via EMA.

`train_jepa_finetune()` : 30 epoche, `batch_size=16`, `lr=1e-3`, `weight_decay=1e-3`. Congela gli encoder e ottimizza solo LSTM + MoE + Gate.

Persistenza: `save_jepa_model()` salva `{model_state_dict, is_pretrained}`. `load_jepa_model()` carica con `weights_only=True` (sicurezza).

SuperpositionLayer — Gating Contestuale (`layers/superposition.py`)

Modulo standalone che implementa un livello lineare con **gating dipendente dal contesto**, usato all'interno del RAP Coach Strategy Layer.

```
class SuperpositionLayer(nn.Module):
    def __init__(self, in_features, out_features, context_dim=METADATA_DIM):
        self.weight = nn.Parameter(empty(out_features, in_features))
        nn.init.kaiming_uniform_(self.weight, a=math.sqrt(5)) # P1-09: Kaiming init
        self.bias = nn.Parameter(zeros(out_features))
        self.context_gate = nn.Linear(context_dim, out_features) # Superposition Controller

    def forward(self, x, context):
        gate = sigmoid(self.context_gate(context)) # [B, out_features]
        self._last_gate_live = gate # Con gradiente (per sparsity loss)
        self._last_gate_activations = gate.detach() # Copia detached (per osservabilità)
        out = F.linear(x, self.weight, self.bias)
        return out * gate # Modulazione contestuale
```

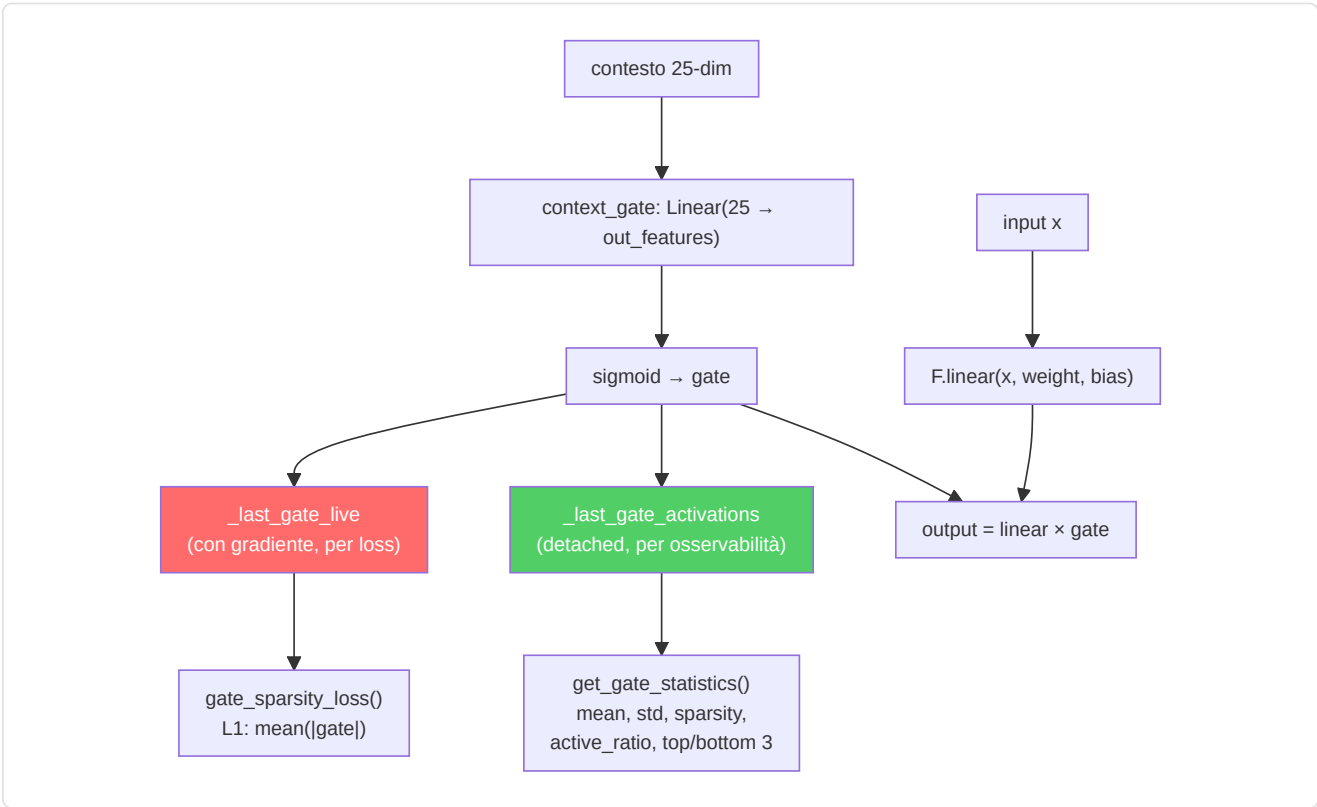
Meccanismo: L'output di ogni neurone viene moltiplicato per un gate sigmoide condizionato sulle feature di contesto (25-dim). Questo permette al modello di "accendere" o "spegnere" neuroni dinamicamente in base alla situazione di gioco.

Inizializzazione Kaiming (P1-09): I pesi vengono inizializzati con `kaiming_uniform_` (distribuzione Kaiming He, 2015) anziché `torch.randn()`. Questa inizializzazione garantisce che la varianza dei pesi sia proporzionale al fan-in del livello, prevenendo la scomparsa o l'esplosione dei gradienti nelle reti profonde. Il parametro `a=math.sqrt(5)` è il valore standard per livelli lineari in PyTorch.

Design dual-tensor (NN-24): Il gate sigmoide viene memorizzato in **due copie separate** durante ogni forward pass:

Tensore	Gradiente	Scopo
<code>_last_gate_live</code>	Sì (mantiene il grafo computazionale)	Usato da <code>gate_sparsity_loss()</code> per backpropagation — il gradiente fluisce attraverso il gate verso <code>context_gate</code>
<code>_last_gate_activations</code>	No (detached)	Usato da <code>get_gate_statistics()</code> per osservabilità — nessun costo di memoria per il grafo

Questa separazione risolve il conflitto tra la necessità di gradienti (per la loss di sparsità) e la necessità di osservabilità leggera (per logging e TensorBoard).



Osservabilità integrata:

Metodo	Ritorno	Descrizione
<code>get_gate_activations()</code>	<code>Tensor</code> o <code>None</code>	Ultime attivazioni del gate (<code>_last_gate_activations</code> , detached)
<code>get_gate_statistics()</code>	<code>Dict[str, float]</code>	Statistiche complete del gate (vedi tabella sotto)
<code>gate_sparsity_loss()</code>	<code>Tensor</code>	Perdita L1 <code>mean()</code>
<code>enable_tracing(interval)</code>	—	Log dettagliato del gate ogni <code>interval</code> passi
<code>disable_tracing()</code>	—	Ripristina intervallo di logging a 100

Campi di `get_gate_statistics()` :

Campo	Tipo	Significato
<code>mean_activation</code>	float	Media delle attivazioni del gate nel batch
<code>std_activation</code>	float	Deviazione standard delle attivazioni
<code>sparsity</code>	float	Frazione di dimensioni con media < 0.1 (più alto = più sparso)
<code>active_ratio</code>	float	Frazione di dimensioni con media > 0.5 (più alto = più attivo)
<code>top_3_dims</code>	List[int]	Le 3 dimensioni del gate più attive
<code>bottom_3_dims</code>	List[int]	Le 3 dimensioni del gate meno attive

Log periodico durante addestramento: Ogni 100 forward pass (configurabile via `enable_tracing(interval)`), logga via logger strutturato: dimensioni attive (gate_mean > 0.5), dimensioni sparse (gate_mean < 0.1) e media complessiva.

Modulo EMA Standalone

L'aggiornamento **Exponential Moving Average** del target encoder è implementato direttamente in

`JEPACoachingModel.update_target_encoder(momentum=0.996)` :

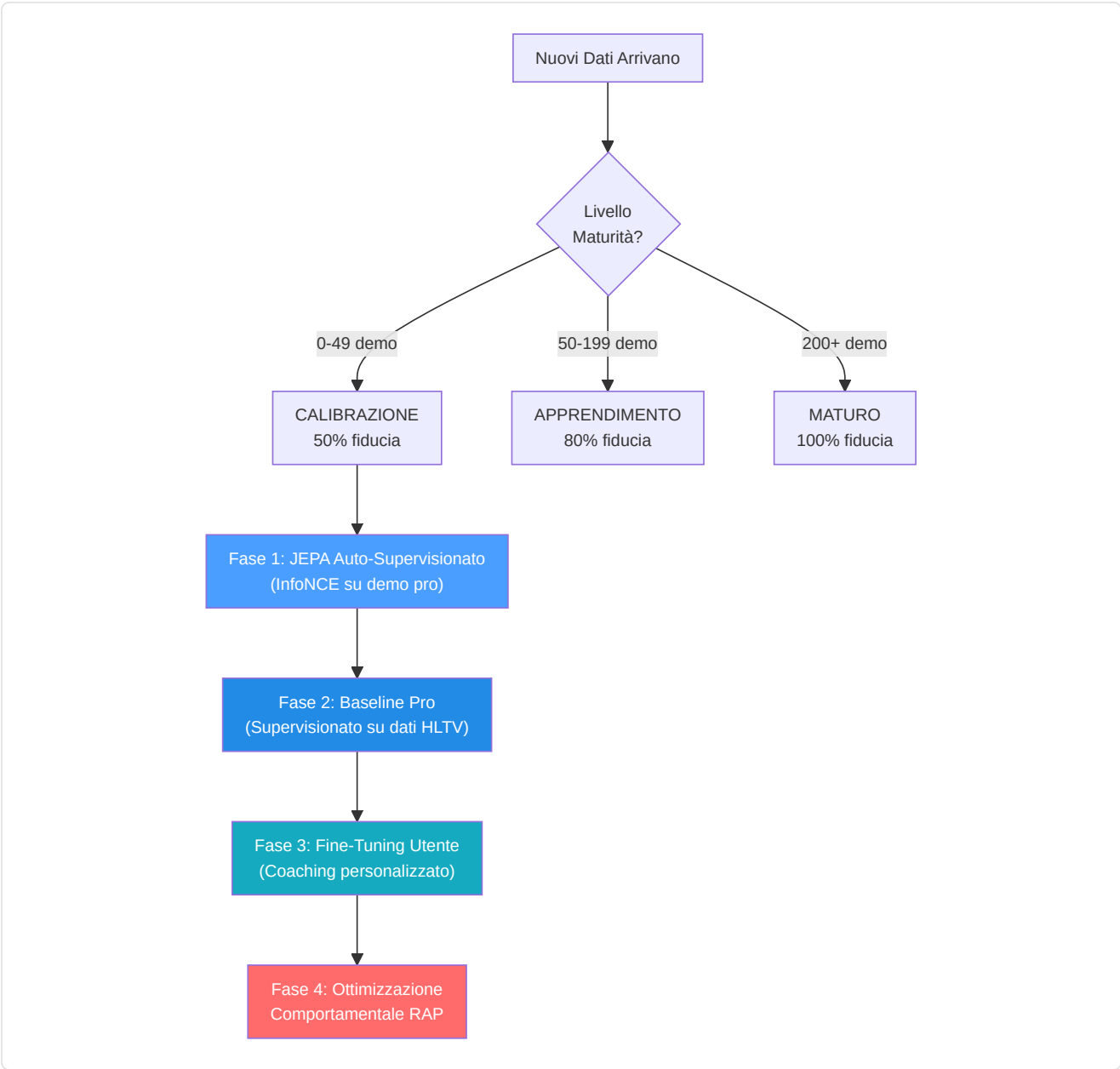
```
with torch.no_grad():
    for param_q, param_k in zip(context_encoder.parameters(), target_encoder.parameters()):
        param_k.data = param_k.data * momentum + param_q.data * (1.0 - momentum)
```

Invarianti:

- L'aggiornamento EMA avviene **sempre dopo** `optimizer.step()` — mai prima, altrimenti i gradienti non sono ancora applicati
- Il target encoder **non riceve mai gradienti** diretti — solo aggiornamenti EMA
- Il momentum 0.996 significa che il target encoder "assorbe" solo lo 0.4% dei pesi dell'encoder online a ogni passo — aggiornamento molto conservativo
- `state_dict()` del modello restituisce tensori **clonati** (`.clone()`) per prevenire aliasing accidentale — un bug reale corretto durante l'audit dove `state_dict()` restituiva riferimenti diretti ai tensori del modello anziché copie, causando corruzione quando il chiamante modificava il dizionario

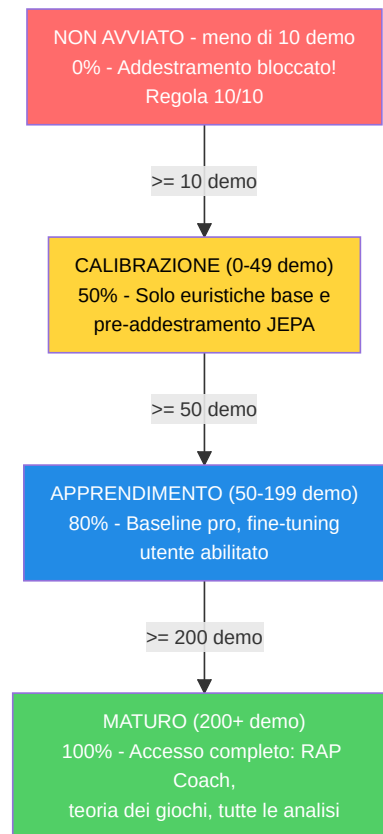
-CoachTrainingManager (Orchestrazione)

Definito in `coach_manager.py` . Questo è il **cervello del processo di formazione**, che gestisce un rigoroso **ciclo di formazione a 3 livelli, basato sulla maturità**, suddiviso in 4 fasi:



Livelli di maturità e moltiplicatori di fiducia:

Livello	Conteggio demo	Moltiplicatore di fiducia	Funzionalità sbloccate
CALIBRAZIONE	0–49	0,50	Euristica di base, pre-addestramento JEPA
APPRENDIMENTO	50–199	0,80	Confronto base professionale, ottimizzazione utente
MATURO	200+	1,00	Coach RAP completo, teoria dei giochi, analisi completa



Prerequisiti (Regola 10/10): Richiede ≥ 10 demo professionali OPPURE (≥ 10 demo utente + account Steam/FACEIT connesso) prima di iniziare qualsiasi allenamento.

Il manager utilizza un **contratto di allenamento** rigoroso con 25 funzionalità (corrispondenti a `METADATA_DIM`).

Problema risolto (ex G-10): `coach_manager.py` ora definisce `TRAINING_FEATURES` con i nomi canonici corretti per tutti i 25 indici, allineati perfettamente con `vectorizer.py`. L'asserzione `len(TRAINING_FEATURES) == METADATA_DIM` è valida e tutti i nomi sono aggiornati. Inoltre, `MATCH_AGGREGATE_FEATURES` definisce le 25 feature aggregate a livello di partita:

`["avg_kills", "avg_deaths", "avg_adr", "avg_hs", "avg_kast", "kill_std", "adr_std", "kd_ratio", "impact_rounds", "accuracy", "econ_rating", "rating", "opening_duel_win_pct", "clutch_win_pct", "trade_kill_ratio", "flash_assists", "positional_aggression_score", "kpr", "dpr", "rating_impact", "rating_survival", "he_damage_per_round", "smokes_per_round", "unused_utility_per_round", "thrusmoke_kill_pct"]`. Entrambe le liste sono validate a runtime: se una delle due ha una lunghezza diversa da `METADATA_DIM`, il modulo solleva `ValueError` al momento dell'importazione.

```

health, armor, has_helmet, has_defuser, equipment_value,
is_crouching, is_scoped, is_blinded,
enemies_visible,
pos_x, pos_y, pos_z,
view_yaw_sin, view_yaw_cos, view_pitch,
z_penalty, kast_estimate, map_id, round_phase,
weapon_class, time_in_round, bomb_planted,
teammates_alive, enemies_alive, team_economy

```

Indici target: `TARGET_INDICES = list(range(OUTPUT_DIM)) = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]` — il modello prevede delta di miglioramento per le prime **10 metriche aggregate** a livello di partita: `["avg_kills", "avg_deaths", "avg_adr", "avg_hs", "avg_kast", "kill_std", "adr_std", "kd_ratio", "impact_rounds", "accuracy"]`.

-TrainingOrchestrator

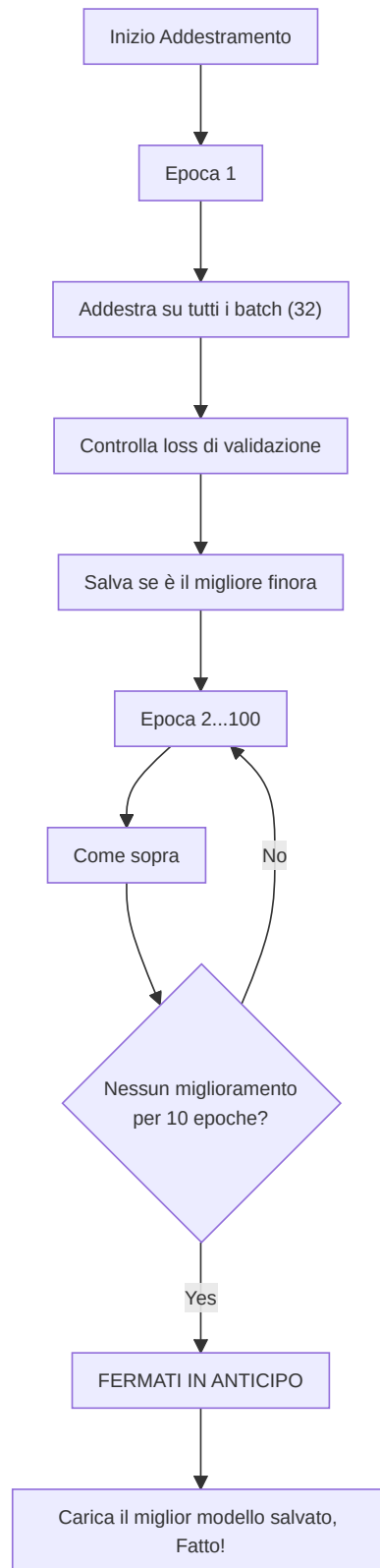
Definito in `training_orchestrator.py`. Ciclo di epoche unificato, convalida, arresto anticipato e checkpoint per i modelli JEPA, VL-JEPA, RAP e RAP Lite.

Parametro	Predefinito	Scopo
<code>model_type</code>	"jepa"	Percorsi verso il trainer JEPA, VL-JEPA, RAP o RAP Lite
<code>max_epochs</code>	100	Limite massimo di allenamento
<code>patience</code>	10	Pazienza nell'arresto anticipato
<code>batch_size</code>	32	Campioni per batch
<code>callbacks</code>	None	Elenco di istanze di <code>TrainingCallback</code> per l'integrazione con Observatory

L'orchestrator si integra con Observatory tramite `CallbackRegistry`. Attiva eventi del ciclo di vita in **5 punti**: `on_train_start` (prima della prima epoca), `on_epoch_start` (inizio di ogni epoca), `on_batch_end` (dopo ogni batch di addestramento, include output di perdita e trainer), `on_epoch_end` (dopo la convalida, include modello e perdite), `on_train_end` (dopo il completamento dell'addestramento o l'interruzione anticipata). Quando non vengono registrate callback, tutte le chiamate `fire()` sono operazioni senza costi. Gli errori di callback vengono rilevati e registrati, senza mai causare l'arresto anomalo del ciclo di addestramento.

Pool negativi cross-match (NN-H-03): L'orchestrator mantiene un pool di feature vectors da batch precedenti (`_neg_pool`, max 500 vettori). I negativi contrastivi vengono campionati da questo pool anziché dal batch corrente, garantendo che i negativi provengano da **partite diverse** e non dalla stessa sequenza temporale di contesto/target. Quando il pool è ancora vuoto (warm-up), il sistema ricade su in-batch sampling. Questo evita falsi negativi: due tick dalla stessa azione di gioco sarebbero troppo simili per essere negativi utili.

Gate di qualità pre-addestramento (P3-D): Prima di iniziare qualsiasi addestramento, l'orchestrator esegue `run_pre_training_quality_check()`. Se il report di qualità non passa (dati insufficienti, distribuzioni anomale, feature mancanti), l'addestramento viene **abortito** con un log di errore esplicativo. Questo impedisce di sprecare GPU su dati che produrrebbero un modello inutile.



-ModelFactory e Persistenza

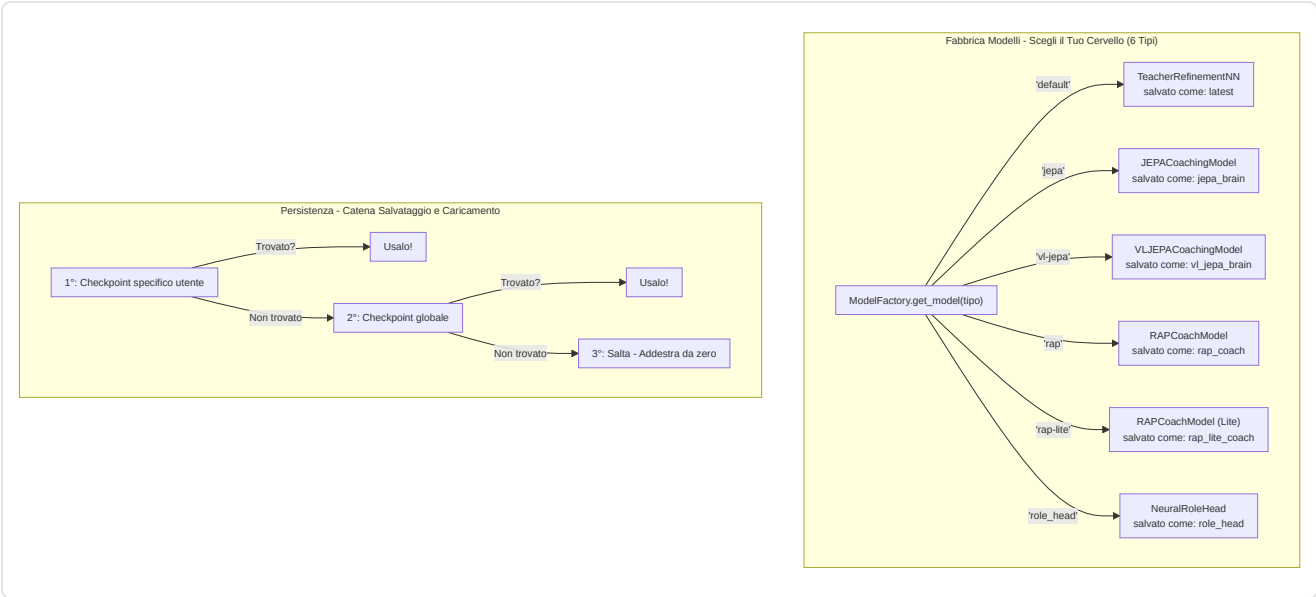
ModelFactory (`factory.py`) fornisce un'istanziatura unificata del modello:

Tipo Costante	Classe Modello	Nome Checkpoint	Impostazioni predefinite di fabbrica
TYPE_LEGACY ("default")	TeacherRefinementNN	"latest"	input_dim=METADATA_DIM(25) , output_dim=OUTPUT_DIM(10) , hidden_dim=HIDDEN_DIM(128)
TYPE_JEPA ("jepa")	JEPACoachingModel	"jepa_brain"	input_dim=METADATA_DIM(25) , output_dim=OUTPUT_DIM(10)
TYPE_VL_JEPA ("vl-jepa")	VLJEPACoachingModel	"vl_jepa_brain"	input_dim=METADATA_DIM(25) , output_dim=OUTPUT_DIM(10)
TYPE_RAP ("rap")	RAPCoachModel	"rap_coach"	metadata_dim=METADATA_DIM(25) , output_dim=10
TYPE_RAP_LITE ("rap-lite")	RAPCoachModel	"rap_lite_coach"	metadata_dim=METADATA_DIM(25) , output_dim=10 , use_lite_memory=True
TYPE_ROLE_HEAD ("role_head")	NeuralRoleHead	"role_head"	input_dim=5 , hidden_dim=32 , output_dim=5

Nota (P1-08): In una versione precedente, la factory utilizzava `output_dim=4` e `hidden_dim=64` per i modelli legacy, creando un disallineamento con `CoachNNConfig`. Questo è stato corretto: ora tutti i modelli di coaching (Legacy, JEPA, VL-JEPA) utilizzano `OUTPUT_DIM = 10` — le prime 10 feature core aggregate su cui il modello predice aggiustamenti delta. `HIDDEN_DIM = 128` è allineato sia in `config.py` che in `factory.py`. Il modello RAP e RAP Lite condividono la stessa classe `RAPCoachModel`, ma RAP Lite attiva `use_lite_memory=True`, sostituendo la memoria LTC-Hopfield con un fallback LSTM puro in PyTorch (utile quando le dipendenze `ncps` / `hflayers` non sono disponibili). Il modello RAP viene importato dal percorso canonico `backend/nn/experimental/rap_coach/model.py` (il vecchio `backend/nn/rap_coach/model.py` è uno shim di reindirizzamento).

StaleCheckpointError: Se le dimensioni di un checkpoint salvato non corrispondono alla configurazione corrente del modello (ad esempio dopo un aggiornamento da `output_dim=4` a `output_dim=10`), il sistema solleva `StaleCheckpointError` anziché caricare silenziosamente pesi incompatibili, prevenendo corruzioni silenziose.

Persistenza (`persistence.py`): Salva/carica modelli con `weights_only=True` (sicurezza — previene esecuzione arbitraria di codice durante il deserializzazione di checkpoint). Implementa un **registro hash SHA-256** per validazione di integrità dei checkpoint: ogni file salvato viene accompagnato da un file sidecar `.sha256` contenente l'hash del checkpoint; al caricamento, il sistema verifica che l'hash corrisponda prima di procedere con il deserializzazione. **Catena di fallback a 3 livelli:** (1) checkpoint specifico dell'utente, (2) checkpoint globale, (3) salta — addestra da zero. Gestisce dimensioni non corrispondenti tramite `StaleCheckpointError`.



-Configurazione (`config.py`)

```
GLOBAL_SEED = 42                # Riproducibilità globale (AR-6, P1-02)
INPUT_DIM = METADATA_DIM = 25   # Vettore canonico a 25 dimensioni (era 19, era legacy 12)
OUTPUT_DIM = 10                 # Le prime 10 feature core su cui il modello predice aggiustamenti
HIDDEN_DIM = 128                # Dimensione nascosta per AdvancedCoachNN / TeacherRefinementNN
BATCH_SIZE = 32
LEARNING_RATE = 0.001
EPOCHS = 50
RAP_POSITION_SCALE = 500.0      # P9-01: Fattore di scala per delta posizione ([-1,1] → unità mondo)
```

Nota: `INPUT_DIM` è importato da `feature_engineering/__init__.py` dove `METADATA_DIM = 25`. `OUTPUT_DIM = 10` definisce il numero di feature su cui il modello produce predizioni di aggiustamento — le prime 10 feature aggregate di match (avg_kills, avg_deaths, avg_adr, avg_hs, avg_kast, kill_std, adr_std, kd_ratio, impact_rounds, accuracy). Questa scelta progettuale concentra la capacità predittiva del modello sulle metriche di prestazione più azionabili, anziché tentare di predire tutte le 25 feature (molte delle quali sono contestuali e non direttamente migliorabili dal giocatore).

`RAP_POSITION_SCALE = 500.0` è il fattore canonico per convertire gli output normalizzati del modello RAP (nell'intervallo [-1, 1]) in spostamenti nelle unità mondo CS2.

Nota architetturale: La `CoachNNConfig` dataclass in `model.py` definisce `output_dim = METADATA_DIM` (25) come default, ma la `ModelFactory` sovrascrive sempre questo valore con `OUTPUT_DIM = 10` durante l'istanziamento. L'output_dim effettivo in produzione per tutti i modelli (Legacy, JEPA, VL-JEPA, RAP, RAP Lite) è quindi **10**, non 25. Storicamente, `OUTPUT_DIM` era 4 (4 metriche selezionate), poi è stato portato a 10 per coprire le feature aggregate più rilevanti.

Gestione dispositivi: `get_device()` implementa una **selezione GPU intelligente a 3 livelli**:

1. **Override utente:** Se configurato `CUDA_DEVICE` (es. "cuda:0" o "cpu"), usa quello
2. **GPU discreta automatica:** `_select_best_cuda_device()` enumera tutti i dispositivi CUDA e seleziona quello con più VRAM, **penalizzando le GPU integrate** (Intel UHD, Iris) tramite keyword matching. Su sistemi multi-GPU (es. Intel UHD + NVIDIA GTX 1650), la GPU discreta vince sempre
3. **Fallback CPU:** Se nessuna GPU CUDA è disponibile

Dimensionamento batch basato sull'intensità ML: `Alto=128`, `Medio=32`, `Basso=8`. Il ritardo di throttling tra batch si adatta: `Alto=0.0s`, `Medio=0.05s`, `Basso=0.2s`.

-Controllo Qualità Dati Pre-Training (`data_quality.py`)

Prima di ogni ciclo di addestramento, il sistema esegue un controllo di qualità sui dati disponibili attraverso `run_pre_training_quality_check()`. Questo gate impedisce l'addestramento su dati insufficienti o corrotti.

Dataclass `DataQualityReport`:

Campo	Tipo	Descrizione
<code>total_tick_rows</code>	<code>int</code>	Righe totali in <code>PlayerTickState</code>
<code>train_rows / val_rows / test_rows</code>	<code>int</code>	Righe per split (TRAIN/VAL/TEST)
<code>zero_position_rate</code>	<code>float</code>	Frazione di tick con posizione (0,0,0) — indicatore di dati corrotti
<code>nan_rate</code>	<code>float</code>	Frazione di tick con valori NaN
<code>round_outcome_distribution</code>	<code>Dict[str,int]</code>	Distribuzione delle classi di outcome per bilanciamento
<code>complete_matches / incomplete_matches</code>	<code>int</code>	Conteggio match completi vs incompleti
<code>issues</code>	<code>List[str]</code>	Lista di problemi rilevati
<code>passed</code>	<code>bool</code>	Verdetto finale (True = addestramento consentito)

5 controlli eseguiti:

1. Conteggio righe totali da `PlayerTickState`
2. Distribuzione per `DatasetSplit` (TRAIN/VAL/TEST) da `PlayerMatchStats`
3. Tasso di posizioni zero — campiona fino a 10.000 tick per efficienza
4. Completezza match via `MatchDataManager`
5. **Verdetto:** fallisce se `total_tick_rows < min_samples` (default: 1000), `zero_position_rate > threshold` (default: 0.10), o `train_rows == 0`

-Training Controller (`training_controller.py`)

Controlla **quando** il Coach può addestrarsi, gestendo deduplicazione demo, diversità dei dati e quota mensile.

Dataclass `TrainingDecision` :

Campo	Tipo	Descrizione
<code>should_train</code>	<code>bool</code>	Se procedere con l'addestramento
<code>reason</code>	<code>str</code>	Motivazione leggibile
<code>diversity_score</code>	<code>float</code>	Score di diversità (0.0–1.0)

Classe `TrainingController` :

- `MIN_DIVERSITY_SCORE = 0.3` — soglia minima di diversità per procedere
- `MAX_DEMOS_PER_MONTH` — quota mensile da settings (default: 10, NN-M-13)

Pipeline di decisione `should_train_on_demo(demo_path, match_stats)` → `TrainingDecision` :

1. **Verifica quota mensile** — `_get_monthly_training_count()` conta `PlayerMatchStats` con `processed_at >= 30` giorni fa
2. **Calcolo diversità** — `_calculate_diversity_score(new_stats)` computa `1 - media_cosine_similarity` contro le ultime 5 partite. Prima partita: score = 1.0
3. **Decisione** — se sotto quota E diversità ≥ 0.3 , `should_train = True`

Calcolo diversità: `_extract_features(stats)` estrae un vettore 6-dim con z-scaling approssimato: kills, deaths, ADR, HS%, utility blind time, opening duel win %. La similarità coseno tra il nuovo vettore e ciascuno degli ultimi 5 previene l'addestramento su dati troppo simili (overfit su una tipologia di partita).

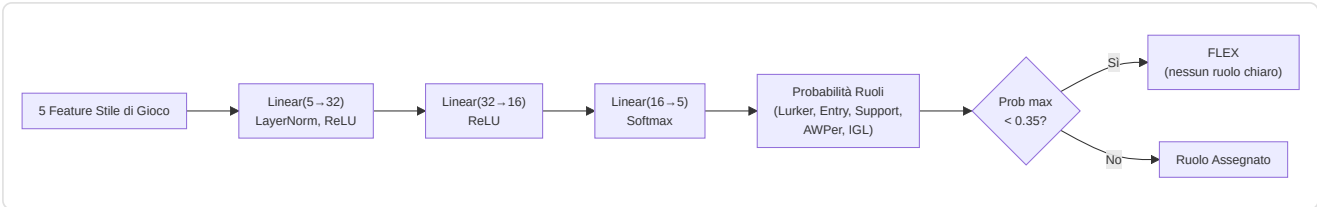
-NeuralRoleHead (MLP per la classificazione dei ruoli)

Definito in `role_head.py`. Un MLP leggero che prevede le probabilità di ruolo dei giocatori in base a 5 parametri di stile di gioco, operando come **opinione secondaria** insieme all'euristica `RoleClassifier`. La logica di consenso in `role_classifier.py` unisce entrambe le opinioni per produrre la classificazione finale.

Architettura:

```
Input (5 caratteristiche) → Linear(5, 32) → LayerNorm(32) → ReLU
→ Linear(32, 16) → ReLU
→ Linear(16, 5) → Softmax → 5 probabilità di ruolo
```

~750 parametri apprendibili. Costo di calcolo minimo, adatto per l'inferenza per partita.



Caratteristiche di input (5 dimensioni):

#	Caratteristica	Sorgente	Intervallo	Significato
0	TAPD	rounds_survived / rounds_played	[0, 1]	Tasso di sopravvivenza — più alto = più passivo/di supporto
1	OAP	entry_frgs / rounds_played	[0, 1]	Aggressività iniziale — più alta = fragger in entrata
2	PODT	was_traded_ratio	[0, 1]	Percentuale di morti scambiate — più alta = scambiate/innescate
3	rating_impact	impact_rating o HLTV 2.0	float	Impatto complessivo sui round
4	aggression_score	positional_aggression_score	float	Tendenza alla posizione avanzata

Ruoli di output (softmax a 5 dimensioni):

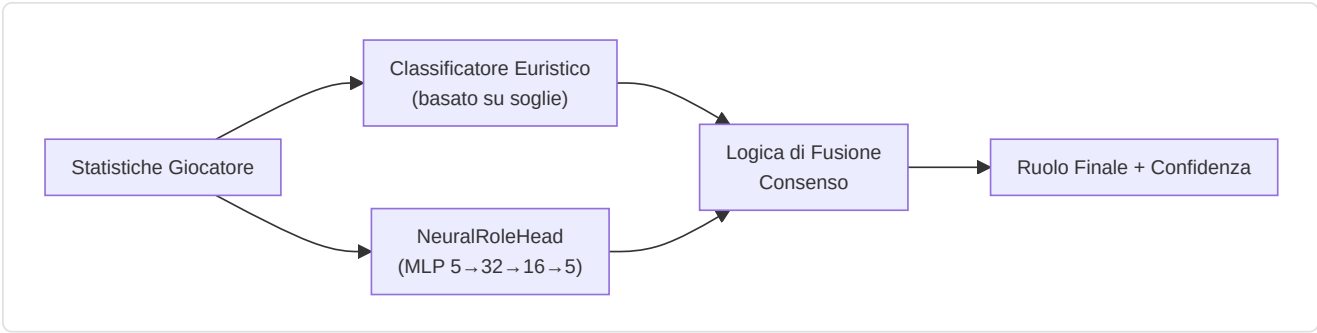
Indice	Ruolo	Descrizione
0	LURKER	Si nasconde dietro le linee nemiche
1	ENTRY_FRAGGER	Primo ad entrare, affronta i duelli iniziali
2	SUPPORT	Ancoraggio del sito, utilizzo delle utilità, scambi
3	AWPER	Specialista cecchino
4	IGL	Leader in gioco, responsabile tattico

Soglia FLEX: Se `max(probabilità) < 0,35` , il giocatore è classificato come **FLEX** (versatile, nessun ruolo dominante). Questo impedisce al modello di forzare un ruolo quando il giocatore è davvero un generalista.

Dettagli addestramento:

Aspetto	Valore
Sconfitta	<code>KLDivLoss(reduction="batchmean")</code> sulle previsioni log-softmax rispetto ai target soft label
Smussamento delle etichette	$\epsilon = 0,02$ (impedisce $\log(0)$, aggiunge la regolarizzazione)
Ottimizzatore	AdamW (lr=1e-3, weight_decay=1e-4)
Arresto anticipato	Pazienza = 15 epoche sulla perdita di convalida
Epoche massime	200
Suddivisione Train/Val	80/20 casuale (dati trasversali, non sequenziali)
Campioni minimi	20 (dalla tabella <code>Ext_PlayerPlaystyle</code>)
Fonte dati	<code>cs2_playstyle_rols_2024.csv</code> → Tabella DB <code>Ext_PlayerPlaystyle</code>
Normalizzazione	Media/std per funzionalità calcolata al momento dell'addestramento, salvata in <code>role_head_norm.json</code>

Consenso con classificatore euristico (`role_classifier.py`):



- **Entrambi d'accordo** → fiducia aumentata di +0,10
- **In disaccordo, margine neurale > 0,1** → vince l'opinione neurale
- **In disaccordo, margine neurale ≤ 0,1** → vince l'opinione euristica
- **Neural non disponibile** (nessun checkpoint o norm_stats) → solo euristica
- **Protezione cold-start** → restituisce FLEX con 0% di confidenza se le soglie non sono state apprese

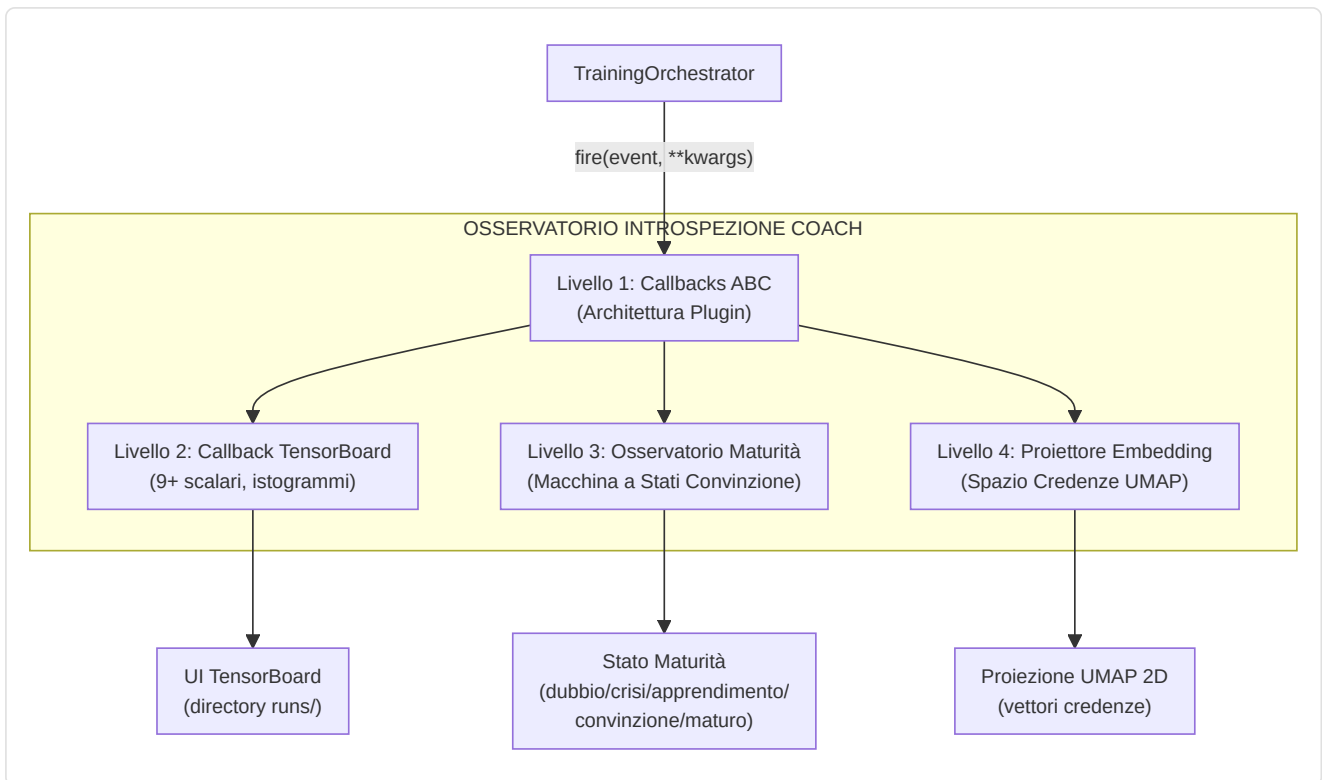
-Coach Introspection Observatory

File: `training_callbacks.py` , `tensorboard_callback.py` , `maturity_observatory.py` , `embedding_projector.py`

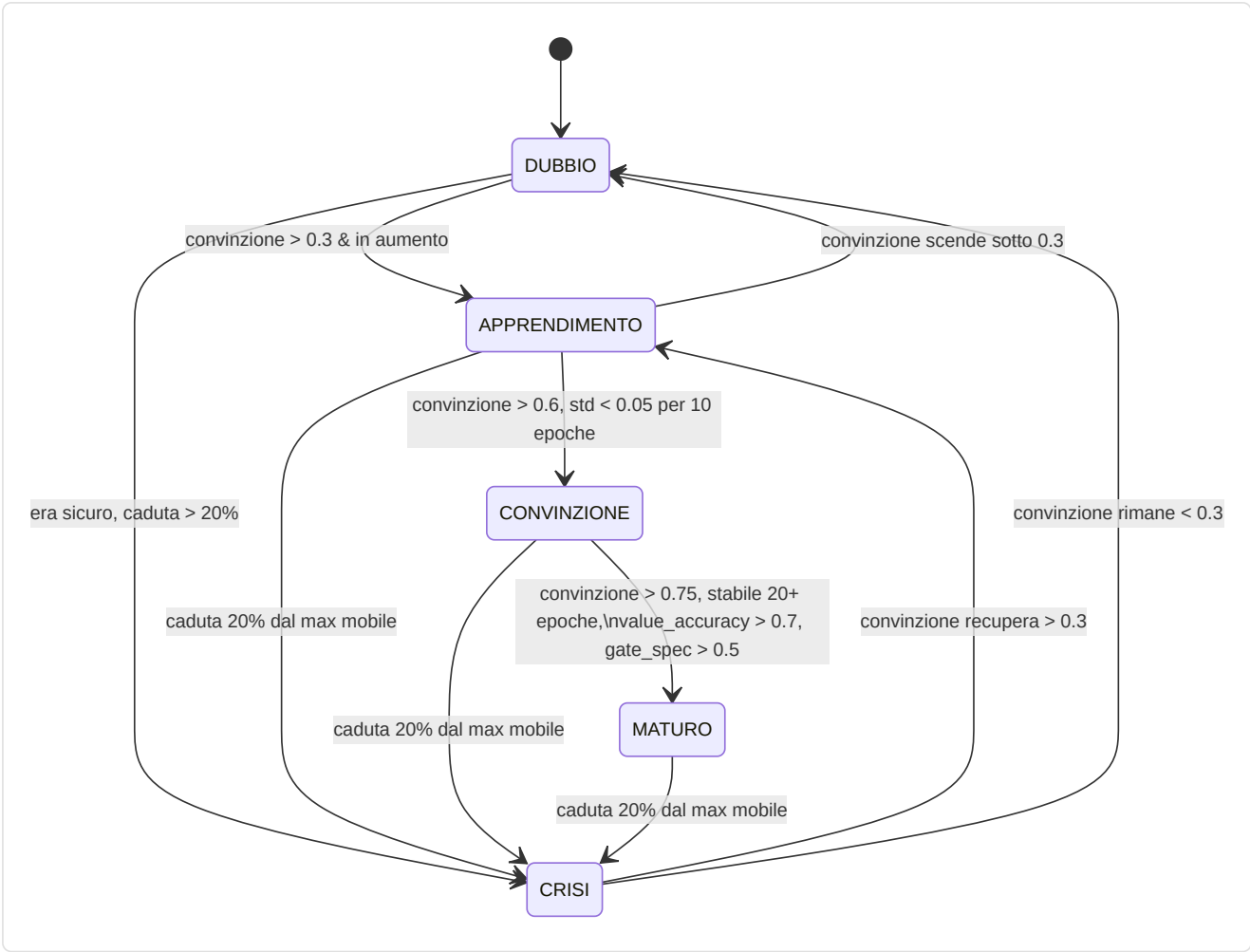
L'Osservatorio è un'architettura di plugin a 4 livelli che strumenta il ciclo di addestramento senza modificare il codice di addestramento principale. Monitora i segnali neurali dell'allenatore durante l'allenamento e li traduce in stati di maturità interpretabili dall'uomo, consentendo a sviluppatori e operatori di capire se il modello è confuso, in fase di apprendimento o pronto per la produzione.

Architettura a 4 livelli:

Livello	File	Scopo	Output chiave
1.Callback ABC	<code>training_callbacks.py</code>	Interfaccia plugin + registro dispatch	<code>TrainingCallback</code> ABC, <code>CallbackRegistry.fire()</code>
2.TensorBoard	<code>tensorboard_callback.py</code>	Registrazione scalare + istogramma	Oltre 9 segnali scalari, istogrammi parametri/grad, istogrammi gate/credenza/concetto
3.Maturità	<code>maturity_observatory.py</code>	Macchina a stati di convinzione	5 segnali → <code>conviction_index</code> → 5 stati di maturità
4.Embedding	<code>embedding_projector.py</code>	Proiezione credenza/concetto UMAP	Figure UMAP 2D interattive (degrado graduale se umap-learn non è installato)

**Maturity State Machine:**

Il `MaturityObservatory` calcola un **indice di convinzione** composito da 5 segnali neurali, lo livella con EMA ($\alpha=0,3$) e classifica il modello in uno dei 5 stati di maturità:



5 Segnali di Maturità:

Segnale	Peso	Intervallo	Cosa Misura	Fonte
<code>belief_entropy</code>	0,25	[0, 1]	Entropia di Shannon del vettore di credenza a 64 dim (più basso = più sicuro)	<code>model._last_belief_batch</code>
<code>gate_specialization</code>	0,25	[0, 1]	<code>1 - mean_gate_activation</code> (più alto = esperti più specializzati)	<code>SuperpositionLayer.get_gate_statistics()</code>
<code>concept_focus</code>	0,20	[0, 1]	<code>1 - entropy(concept_embedding_norms)</code> (entropia più bassa = focalizzato)	<code>model.concept_embeddings</code>
<code>value_accuracy</code>	0,20	[0, 1]	<code>1 - (val_loss / initial_val_loss)</code> (più alto = migliore calibrazione)	Ciclo di convalida
<code>role_stability</code>	0,10	[0, 1]	Coerenza della convinzione nelle epoche recenti (<code>1 - std*5</code>)	Cronologia autoreferenziale

Formula di convinzione:

```
indice_convinzione = 0,25 × (1 - entropia_credenza)
+ 0,25 × specializzazione_gate
+ 0,20 × focus_concetto
+ 0,20 × accuratezza_valore
+ 0,10 × stabilità_ruolo

punteggio_maturità = EMA(indice_convinzione, α=0,3)
```

Soglie di stato:

Stato	Condizione
DUBBIO	<code>conviction < 0,3</code>
CRISI	<code>conviction</code> scende > 20% dal massimo mobile entro 5 epoche
APPRENDIMENTO	<code>conviction ∈ [0,3, 0,6]</code> e in aumento
CONVICTION	<code>conviction > 0,6</code> , stabile (<code>std < 0,05</code> su 10 epoche)
MATURE	<code>conviction > 0,75</code> , stabile per oltre 20 epoche, <code>value_accuracy > 0,7</code> , <code>gate_specialization > 0,5</code>

MaturitySnapshot `dataclass`: Ogni epoca, l'Osservatorio produce un'istantanea immutabile:

Campo	Tipo	Descrizione
<code>epoch</code>	int	Numero dell'epoca
<code>timestamp</code>	datetime	Momento della registrazione
<code>belief_entropy</code>	float	Entropia di Shannon del vettore credenze
<code>gate_specialization</code>	float	Specializzazione degli esperti
<code>concept_focus</code>	float	Focalizzazione sui concetti di coaching
<code>value_accuracy</code>	float	Accuratezza delle predizioni di valore
<code>role_stability</code>	float	Stabilità della classificazione dei ruoli
<code>conviction_index</code>	float	Indice composito pesato
<code>maturity_score</code>	float	Punteggio EMA livellato ($\alpha=0.3$)
<code>state</code>	str	Stato corrente (DUBBIO/CRISI/APPRENDIMENTO/CONVINZIONE/MATURO)

Estrazione dei 5 segnali neurali — come vengono calcolati:

Segnale	Metodo	Fonte dati	Calcolo
<code>belief_entropy</code>	<code>_compute_belief_entropy()</code>	<code>model._last_belief_batch</code>	$\text{softmax}(\text{belief}) \rightarrow \text{Shannon entropy} / \log(\text{dim}) \rightarrow 1 - \text{normalizzata}$
<code>gate_specialization</code>	<code>_compute_gate_specialization()</code>	<code>strategy.superposition.get_gate_statistics()</code>	$1 - \text{mean_activation}$ (più alto = esperti più specializzati)
<code>concept_focus</code>	<code>_compute_concept_focus()</code>	<code>model.concept_embeddings.weight</code>	norme L2 $\rightarrow \text{softmax} \rightarrow 1 - \text{entropy}$ (più basso = più focalizzato)
<code>value_accuracy</code>	<code>_compute_value_accuracy()</code>	Ciclo di validazione	$1 - (\text{val_loss} / \text{initial_val_loss})$, clamped [0, 1]
<code>role_stability</code>	<code>_compute_role_stability()</code>	Cronologia recente	$1 - \text{std}(\text{ultimi } 10 \text{ conviction_index}) \times 5$, clamped [0, 1]

API pubblica: `current_state` (proprietà $\rightarrow$ stringa stato), `current_conviction` (proprietà $\rightarrow$ float), `get_timeline()` ($\rightarrow$ lista di `MaturitySnapshot` per esportazione/grafici).

Integrazione TensorBoard: Ogni `on_epoch_end()` registra **7 scalari** su TensorBoard:


```
maturity/belief_entropy, maturity/gate_specialization, maturity/concept_focus,
maturity/value_accuracy, maturity/role_stability,
maturity/conviction_index, maturity/maturity_score
```

Più un log testuale dello stato corrente via logger strutturato.

Garanzie di progettazione:

- **Impatto zero se disabilitato:** Quando non vengono registrate callback, tutte le chiamate `CallbackRegistry.fire()` sono no-op. Nessuna allocazione di memoria, nessun overhead di calcolo.
- **Isolamento degli errori:** Ogni callback è sottoposto individualmente a try/except-wrapping. Un errore di scrittura di TensorBoard o di calcolo UMAP non causa mai l'arresto anomalo del ciclo di training: l'errore viene registrato e il training continua.
- **Componibile:** È possibile aggiungere nuove callback sottoclassando `TrainingCallback` e registrandosi con `CallbackRegistry.add()`. Non è necessario modificare il codice di training.

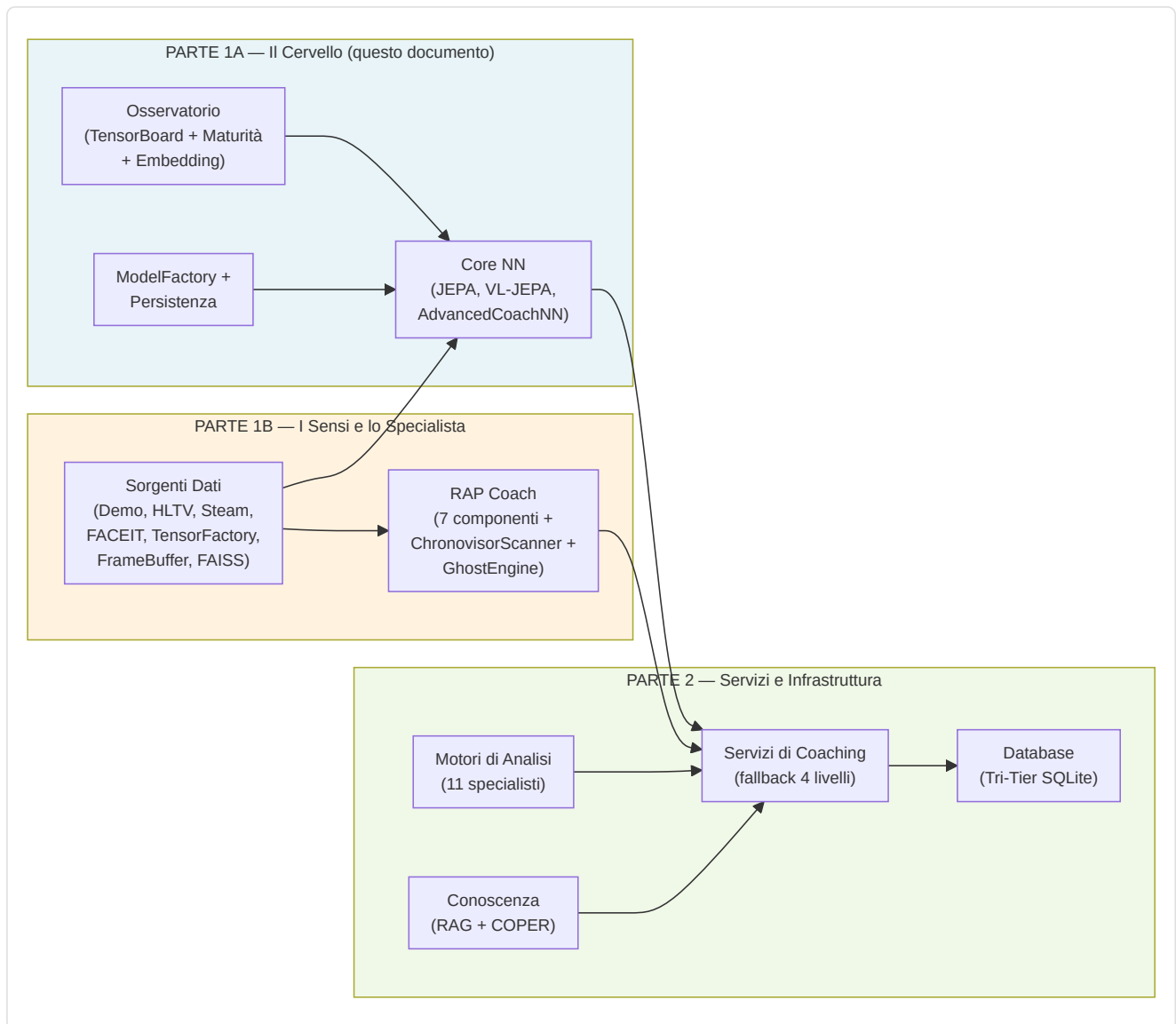
Integrazione CLI: Avviato tramite `run_full_training_cycle.py` con i flag:

- `--no-tensorboard` — disabilita il callback di TensorBoard
- `--tb-logdir <percorso>` — imposta la directory di log di TensorBoard (predefinito: `runs/`)
- `--umap-interval <N>` — proiezione UMAP ogni N epoche (predefinito: 10)

Riepilogo della Parte 1A — Il Cervello

La Parte 1A ha documentato il **nucleo cognitivo** del sistema di coaching — l'intero sottosistema di reti neurali che costituisce il "cervello" del coach:

Componente	Ruolo	Dettagli Chiave
AdvancedCoachNN	Coaching supervisionato base	LSTM a 2 livelli + 3 esperti MoE, input 25-dim, output 10-dim
JEPA	Pre-addestramento auto-supervisionato	Encoder online/target (EMA $\tau=0.996$), InfoNCE contrastivo
VL-JEPA	Allineamento visione-linguaggio	16 concetti di coaching in 5 dimensioni tattiche
SuperpositionLayer	Gating contestuale	Modulazione dipendente dal contesto 25-dim con osservabilità integrata
CoachTrainingManager	Orchestrazione addestramento	3 livelli di maturità (CALIBRAZIONE → APPRENDIMENTO → MATURO)
TrainingOrchestrator	Ciclo di epoche unificato	Early stopping, checkpoint, callback, pool negativi cross-match, quality gate
ModelFactory	Istanziamento modelli	6 tipi di modello (+ RAP Lite) con persistenza e fallback
NeuralRoleHead	Classificazione ruoli	MLP 5 → 32 → 16 → 5, consenso con euristica
MaturityObservatory	Introspezione addestramento	5 segnali → indice di convinzione → 5 stati di maturità



Continua nella Parte 1B — I Sensi e lo Specialista: RAP Coach Model, ChronovisorScanner, GhostEngine, Sorgenti Dati (Demo Parser, HLTV, Steam, FACEIT, TensorFactory, FrameBuffer, FAISS, Round Context)